

T.C.
ESKİŞEHİR OSMANGAZİ ÜNİVERSİTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
2025-2026 Eğitim-Öğretim Yılı



152118640- ENGINEERING RESEARCH ON CYBER SECURITY

1. ARASINAV RAPORU

Proje Başlığı

“PHOENIX (Phishing Hazard Observation and
Evaluation using Neural Intelligence eXtension) ”

Projeyi Hazırlayanlar:

Dide ŞENGÜL 152120221041

Ata Can ÖZCİHAN 152120221103

Danışman:

Dr. Öğr. Üyesi İLKER ÖZÇELİK

A. PLANLAMA	4
A.1. Özet (Abstract) ve Anahtar Kelimeler (Keywords)	4
A.2. Bilgi Gereksinim Belirleme, Problemin Tanımlanması	5
A.2.1. Amaç	5
A.2.2. Konu ve Kapsam	5
A.2.3. Literatür Özeti	6
A.3. Beklenen Fayda	6
A.3.1. Özgün Değer	6
A.3.2. Yaygın Etki / Katma Değer	7
A.4. Yöntem	7
A.4.1. Araştırma Tasarımı ve Yaklaşım	7
A.4.2. Derin Öğrenme Modeli	7
A.4.3. Backend API Geliştirme	8
A.4.4. Chrome Tarayıcı Eklentisi	8
A.5. Araştırma Olanakları	8
A.5.1. Donanım Olanakları	8
A.5.2. Yazılım ve Araçlar	8
A.5.3. Veri Kaynakları	8
A.6. Çalışma Takvimi	9
A.6.1. İş Zaman Çizelgesi	9
A.6.2. Kim Ne İş Yapacak	9
B. ANALİZ	10
B.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri	10
B.1.1. Yüz Yüze Görüşme	10
B.1.2. Yazılı Basılı Belge İnceleme	10
B.1.3. Anket	11
B.1.4. Gözlem	13
B.1.5. Veri Akış Şemaları	13
B.1.5.1. Kavramsal Veri Akış Şeması	13
B.1.5.2. Mantıksal Veri Akış Şeması	13
B.1.5.3. Fiziksel Veri Akış Şeması	14
B.1.6. Olay Tabloları, Durum Formları, İşlevsel Analiz Raporu, İş Akış Şeması	14
B.1.6.1 Durum Formları	15
B.1.6.2 İşlevsel Analiz Raporu	15
B.1.6.4 İş Akış Şeması	16
B.2. Sistem Gereksinimleri	17
B.2.1. İşlevsel Gereksinimler	17
B.2.2. Sistem ve Kullanıcı Arayüzleri ile ilgili Gereksinimler	17
B.2.3. Veriyle İlgili Gereksinimler	18
B.2.4. Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri	18
B.2.4.1. Kullanıcılar ve İnsan Faktörü Gereksinimleri	18
B.2.4.2. Güvenlik Gereksinimleri	19
B.2.5. Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler	19
B.2.5.1. Teknik Gereksinimler	19
B.2.5.2. Fiziksel Gereksinimler:	20
C. TASARIM	20
C.1. Sistem Tasarımı	20

C.2. Kullanıcı ve Sistem Arayüzü Tasarımları	21
C.2.1. Chrome Eklentisi Popup Arayüzü	21
C.2.2. Ayarlar ve Dashboard Sayfası	22
C.2.3. Manuel URL Analizi	23
C.3. Test Tasarımı	24
C.3.1. Gereksinim Analizlerinden Test Hedeflerinin Belirlenmesi	24
C.3.2. Fonksiyonel Test Tasarımı	24
C.3.3. Performans Test Tasarımı	25
C.4. Yazılım Tasarımı	25
C.4.1. Gereksinime Bağlı Tasarım Kalıpları Seçimi	25
C.4.2. UML Kullanılarak Tasarım Diyagramları	26
C.5. Veri Tabanı Tasarımı	28
C.5.1. Veri Tabanı Tabloları Dökümü	28
C.5.2. E/R Diyagramı	29
C.5.3. Stored Procedures ve Triggers	30
C.6. Donanım Tasarımı	30
C.6.1. Sistem Mekanik Bileşen Tasarımlarının Verilmesi	30
C.6.2. Elektronik Tasarım	30
C.6.3. Gömülü Yazılım Tasarımları	30

A. PLANLAMA

A.1. Özet (Abstract) ve Anahtar Kelimeler (Keywords)

ÖZET

İnternet kullanımının yaygınlaşmasıyla birlikte ortalama (phishing) saldırıları, kullanıcıların kişisel bilgilerini, finansal verilerini ve hesap erişimlerini ele geçirmek amacıyla en sık kullanılan siber saldırı yöntemlerinden biri haline gelmiştir. Ortalama saldırıları genellikle sahte web siteleri aracılığıyla gerçekleştirilmekte ve kullanıcıların gerçek hizmetlere benzer arayüzler üzerinden kimlik bilgilerini girmeleri sağlanmaktadır. Günümüzde bu tür saldırılar giderek daha karmaşık hale gelmekte ve geleneksel kara liste (blacklist) veya kural tabanlı güvenlik yaklaşımlarının bu tehditleri tespit etme başarısı sınırlı kalabilmektedir. Bu nedenle, ortalama web sitelerinin daha doğru ve hızlı bir şekilde tespit edilmesi için yapay zeka ve özellikle derin öğrenme tabanlı yöntemlere olan ilgi artmaktadır.

Bu bitirme projesinde PHOENIX (Phishing Hazard Observation and Evaluation using Neural Intelligence eXtension) adlı bir sistem geliştirilmesi amaçlanmaktadır. PHOENIX, derin öğrenme tabanlı bir ortalama web sitesi tespit modeli ile çalışan ve kullanıcıyı gerçek zamanlı olarak uyararak bir tarayıcı eklentisi olarak tasarlanacaktır. Sistem, ziyaret edilen web sitelerinin URL yapısı ve sayfa özelliklerinden elde edilen verileri analiz ederek ilgili web sayfasının ortalama saldırısı içerip içermediğini belirlemeye çalışacaktır. Modelin eğitimi için ortalama ve meşru web sitelerini içeren bir veri seti kullanılacak ve veri ön işleme, özellik çıkarımı ve model eğitimi adımları uygulanacaktır. Eğitim sürecinin ardından geliştirilen model doğruluk, precision, recall ve F1-score gibi performans metrikleri kullanılarak değerlendirilecektir.

Geliştirilecek tarayıcı eklentisi sayesinde kullanıcılar şüpheli web sitelerine girdiklerinde anlık olarak uyarılabilecek ve potansiyel ortalama saldırılarından korunabileceklerdir. Bu yönüyle PHOENIX sistemi hem makine öğrenmesi hem de siber güvenlik alanlarını birleştiren uygulamalı bir çözüm sunmayı hedeflemektedir. Projenin temel amacı, derin öğrenme yöntemlerinin ortalama saldırılarının tespitindeki etkinliğini incelemek ve kullanıcı güvenliğini artırmaya yönelik pratik bir güvenlik aracı geliştirmektir.

Anahtar Kelimeler: Ortalama saldırısı, siber güvenlik, derin öğrenme, makine öğrenmesi, phishing tespiti, tarayıcı eklentisi

ABSTRACT

With the widespread use of the Internet, phishing attacks have become one of the most common cyber threats aimed at stealing users' personal information, financial data, and account credentials. Phishing attacks are typically carried out through fraudulent websites that imitate legitimate services and trick users into entering sensitive information. As these attacks become increasingly sophisticated, traditional security approaches such as blacklist-based or rule-based detection methods often struggle to identify newly created phishing websites effectively. For this reason, artificial intelligence and especially deep learning-based techniques have gained significant attention for improving the detection accuracy of phishing websites.

In this graduation project, a system called PHOENIX (Phishing Hazard Observation and Evaluation using Neural Intelligence eXtension) is proposed and developed. PHOENIX is designed as a browser extension supported by a deep learning-based phishing detection model that provides real-time warnings to users. The system analyzes various characteristics of visited websites, particularly URL structures and webpage-related features, to determine whether the webpage is potentially malicious or legitimate. A dataset containing both phishing and legitimate websites will be used to train the model, and several steps such as data preprocessing, feature extraction, and model training will be applied during the development process. After the training phase, the model will be evaluated using standard performance metrics including accuracy, precision, recall, and F1-score.

The developed browser extension aims to enhance user security by providing instant alerts when suspicious websites are detected. In this way, PHOENIX offers a practical cybersecurity solution that combines machine learning techniques with real-world applications. The main objective of the project is to investigate the effectiveness of deep learning approaches in detecting phishing attacks while developing a functional tool that helps users avoid malicious websites and improve overall web security.

Keywords: Phishing attack, cybersecurity, deep learning, machine learning, phishing detection, browser extension

A.2. Bilgi Gereksinim Belirleme, Problemin Tanımlanması

İnternet kullanımının hızla yaygınlaşmasıyla birlikte siber saldırılar da giderek daha sofistike bir hal almaktadır. Oltalama (phishing) saldırıları, bu tehditler arasında en yaygın ve etkili olanlardan biri olmaya devam etmekte; kullanıcıların kimlik bilgilerini, finansal verilerini ve kişisel hesap erişimlerini ele geçirmek amacıyla sıklıkla kullanılmaktadır. Saldırganlar, meşru web sitelerini birebir taklit eden sahte sayfalar oluşturarak kullanıcıları yanıltmakta ve bu sayfalara hassas bilgilerini girmeleri için yönlendirmektedir.

Mevcut güvenlik çözümleri incelendiğinde, kara liste (blacklist) tabanlı yaklaşımların yalnızca daha önce raporlanmış olan oltalama sitelerini engelleyebildiği görülmektedir. Oysa saldırırganlar sürekli yeni alan adları ve sunucular kullanarak bu listeleri kolaylıkla aşabilmektedir. Kural tabanlı sistemler ise statik yapıları nedeniyle değişen saldırı tekniklerine karşı güncelliğini yitirmektedir. Bu durum, özellikle aşağıdaki sorunları beraberinde getirmektedir:

- Yeni oluşturulan oltalama sitelerinin kara listelere eklenmeden önce kullanıcılara zarar verebilmesi
- Kural tabanlı sistemlerin görsel olarak meşru sitelere benzeyen sahte siteleri ayırt edememesi
- Teknik bilgisi sınırlı bireysel kullanıcıların şüpheli siteleri manuel olarak fark edememesi
- Mevcut tarayıcı güvenlik araçlarının yüksek yanlış pozitif oranlarıyla kullanıcı deneyimini olumsuz etkilemesi

Bu problemlerin üstesinden gelmek amacıyla gerçekleştirilen araştırmalar, literatür taraması ve veri analizi süreçleri sonucunda, makine öğrenmesi ve derin öğrenme tabanlı bir çözümün geliştirilmesinin gerekli olduğu sonucuna varılmıştır. Vrbancıç ve ark. (2020) tarafından hazırlanan ve 111 URL tabanlı özellik içeren veri seti incelenmiş; bu veri setinin derin öğrenme modellerinin eğitimi için uygun bir kaynak oluşturduğu değerlendirilmiştir. Yapılan analizler doğrultusunda, kullanıcıya gerçek zamanlı uyarı sunan, tarayıcıya entegre edilebilen ve derin öğrenme tabanlı bir oltalama tespit sisteminin geliştirilmesi gerekliliği ortaya konmuştur.

A.2.1. Amaç

Bu projenin temel amacı, derin öğrenme yöntemlerini kullanarak oltalama web sitelerini gerçek zamanlı olarak tespit edebilen ve kullanıcıyı anlık biçimde uyarı veren bir tarayıcı eklentisi geliştirmektir. PHOENIX sistemi, ziyaret edilen web sitesine ait URL yapısı ve sayfa özelliklerinden elde edilen verileri analiz ederek söz konusu sitenin meşru mu yoksa oltalama amaçlı mı olduğunu sınıflandıracaktır.

Proje kapsamında aşağıdaki hedefler belirlenmiştir:

- Oltalama ve meşru web sitelerine ait URL tabanlı özelliklerin çıkarılması ve ön işleme adımlarının uygulanması
- Derin öğrenme tabanlı bir sınıflandırma modelinin tasarlanması, eğitilmesi ve doğruluk, precision, recall ve F1-score gibi performans metrikleri kullanılarak değerlendirilmesi
- Eğitilen modelin bir Google Chrome tarayıcı eklentisine entegre edilerek kullanıcıya gerçek zamanlı tehdit bildirimi sunulması
- Geleneksel kara liste tabanlı yaklaşımların yetersiz kaldığı durumlarda dahi yüksek doğruluk oranıyla çalışabilen, kullanıcı dostu ve pratik bir güvenlik aracının ortaya konması

Bu proje, yalnızca akademik bir araştırma olmanın ötesinde, bireysel internet kullanıcılarının oltalama saldırılarına karşı korunmasına doğrudan katkı sağlamayı hedefleyen uygulamalı bir siber güvenlik çözümü olarak tasarlanmıştır.

A.2.2. Konu ve Kapsam

Bu çalışma, oltalama web sitelerinin tespitine yönelik derin öğrenme tabanlı bir sınıflandırma modelinin geliştirilmesini ve bu modelin bir Google Chrome tarayıcı eklentisi aracılığıyla son kullanıcıya sunulmasını kapsamaktadır. Sistem, URL yapısından elde edilen sayısal özellikler temelinde çalışmakta olup ziyaret edilen her web sayfasını arka planda analiz ederek kullanıcıyı gerçek zamanlı olarak bilgilendirmektedir.

PHOENIX sisteminin temel bileşenleri ve kapsadığı konular şu şekilde özetlenebilir:

- **Veri Seti:** Model eğitimi için Vrbancıç ve ark. (2020) tarafından hazırlanan veri seti kullanılacaktır. Bu veri seti; URL özellikleri, alan adı bilgileri, yönlendirme metrikleri ve harici servis verileri dahil olmak üzere 111 özelliğten oluşmakta ve 58.645 ile 88.647 web sitesi örneği içeren iki farklı varyasyon sunmaktadır. Veri seti, PhishTank kaynaklı doğrulanmış oltalama URL'leri ile Alexa üst sıra siteleri ve topluluk etiketli meşru URL'lerden derlenmiştir.

- **Özellik Grupları:** Veri setinde yer alan özellikler altı ana grupta incelenmektedir. Bunlar; URL genelindeki karakter sayılarına dayalı özellikler, alan adı özellikleri, dizin (directory) özellikleri, dosya adı özellikleri, URL parametre özellikleri ve URL çözümleme ile harici servis metriklerine dayalı özelliklerdir.
- **Model Geliştirme:** Çıkarılan özellikler kullanılarak bir derin öğrenme modeli tasarlanacak, eğitilecek ve test edilecektir. Model performansı doğruluk (accuracy), kesinlik (precision), duyarlılık (recall) ve F1-score metrikleriyle değerlendirilecektir.
- **Tarayıcı Eklentisi:** Eğitilen model, Google Chrome Manifest V3 standardına uygun bir tarayıcı eklentisine entegre edilecektir. Eklenti, kullanıcı bir web sitesini ziyaret ettiğinde ilgili URL bilgilerini arka uç (backend) servise iletecek, modelden alınan tahmin sonucuna göre kullanıcıya görsel bir uyarı gösterecektir.
- **Kapsam Dışında Kalan Konular:** Bu proje kapsamında e-posta tabanlı ortalama tespiti, görsel benzerlik analizi ve HTTPS sertifika doğrulaması gibi konular ele alınmayacaktır. Sistem yalnızca URL ve web sitesi özelliklerine dayalı sınıflandırma üzerine odaklanmaktadır.

A.2.3. Literatür Özeti

Oltalama web sitelerinin tespiti, makine öğrenmesi ve derin öğrenme alanında kapsamlı biçimde incelenen bir araştırma konusudur. Bu alanda yapılan çalışmalar; kullanılan özellik grupları, sınıflandırma algoritmaları ve sistem mimarileri bakımından çeşitlilik göstermektedir.

Mohammad ve ark. (2012), ortalama web sitelerinin tespitinde URL tabanlı özelliklerin, HTML ve JavaScript kaynaklı özelliklerin ve alan adı kayıt bilgilerinin birlikte kullanımının sınıflandırma başarımını önemli ölçüde artırdığını göstermiştir. Çalışmada, otomatik özellik çıkarım süreçleri üzerinde durulmuş ve farklı özellik gruplarının model performansına katkısı karşılaştırmalı olarak değerlendirilmiştir.

Abdelhamid ve ark. (2014), ilişkilendirme kuralı madenciliğine (associative classification) dayalı bir ortalama tespit yöntemi önermiştir. Çalışmada geliştirilen yaklaşımın, geleneksel makine öğrenmesi yöntemleriyle kıyaslandığında rekabetçi doğruluk oranları sunduğu ortaya konmuştur. Ayrıca özellik seçiminin model başarımı üzerindeki belirleyici rolü vurgulanmıştır.

Vrbančić ve ark. (2019), derin sinir ağlarının ortalama web sitesi sınıflandırmasındaki etkinliğini incelemiş ve sürü zekâsı (swarm intelligence) tabanlı parametre optimizasyonunun model performansını iyileştirdiğini göstermiştir. Aynı araştırmacılar tarafından 2020 yılında yayımlanan çalışmada ise PhishTank ve Alexa kaynaklarından derlenen, 111 URL tabanlı özellik içeren ve iki farklı boyutta sunulan bir veri seti kamuya açık hale getirilmiştir. 58.645 ve 88.647 web sitesi örneği içeren bu veri seti, literatürde yaygın biçimde kullanılan bir kıyaslama kaynağı haline gelmiş olup bu projede de temel veri kaynağı olarak kullanılacaktır.

Güncel araştırmalar incelendiğinde, yalnızca URL tabanlı özelliklerin kullanıldığı çalışmalardan sayfa içeriğini, görsel benzerlikleri ve DNS kayıtlarını bir arada değerlendiren çok katmanlı yaklaşımlara doğru belirgin bir eğilim gözlemlenmektedir. Bununla birlikte, bu gelişmiş yöntemlerin son kullanıcıya yönelik pratik tarayıcı eklentileri biçiminde hayata geçirilmesi konusunda literatürde hâlâ önemli bir boşluk bulunmaktadır. PHOENIX projesi, derin öğrenme tabanlı tespit modelini gerçek dünya kullanımına uygun bir tarayıcı eklentisiyle buluşturarak bu boşluğu doldurmayı hedeflemektedir.

A.3. Beklenen Fayda

A.3.1. Özgün Değer

Bu çalışma, derin öğrenme tabanlı ortalama tespitini gerçek zamanlı çalışan bir tarayıcı eklentisiyle birleştirerek literatürdeki önemli bir boşluğu doldurmayı hedeflemektedir. Mevcut akademik çalışmaların büyük çoğunluğu sınıflandırma modellerinin geliştirilmesi ve performans değerlendirmesiyle sınırlı kalmakta; bu modellerin son kullanıcıya yönelik işlevsel bir araca dönüştürülmesi konusu yeterince ele alınmamaktadır. PHOENIX sistemi, bu iki alanı bir arada ele alarak hem teknik hem de kullanıcı odaklı bir çözüm sunmaktadır.

Sistemin öne çıkan özgün değerleri şunlardır:

- **Gerçek Zamanlı Koruma:** PHOENIX, kullanıcı bir web sitesini ziyaret ettiği anda arka planda analiz gerçekleştirerek ortalama girişimini tespit etmekte ve kullanıcıyı anında uyarmaktadır. Bu yaklaşım, kara liste tabanlı sistemlerin aksine daha önce hiç raporlanmamış yeni ortalama sitelerini de tespit edebilme kapasitesi sunmaktadır.

- **URL Tabanlı Çok Boyutlu Özellik Analizi:** Sistem, yalnızca alan adı kontrolüyle sınırlı kalmayıp URL geneli, alan adı, dizin, dosya adı, parametre ve harici servis metrikleri dahil olmak üzere 111 farklı özelliği eş zamanlı olarak analiz etmektedir. Bu çok boyutlu yaklaşım, tek özellik grubuna dayanan yöntemlere kıyasla daha yüksek tespit başarımı sağlamaktadır.
- **Kullanıcı Dostu Entegrasyon:** Sistem, kullanıcıların ek bir yazılım kurmasına gerek kalmadan doğrudan tarayıcı üzerinden çalışmaktadır. Teknik bilgisi bulunmayan bireysel kullanıcılar bile herhangi bir yapılandırma yapmadan sistemden yararlanabilmektedir.
- **Açık Kaynak Veri Setiyle Tekrarlanabilirlik:** Model eğitimi için kamuya açık ve akademik çevrelerde yaygın biçimde kabul görmüş bir veri seti kullanılmakta olup bu durum çalışmanın tekrarlanabilirliğini ve kıyaslanabilirliğini artırmaktadır.

A.3.2. Yaygın Etki / Katma Değer

PHOENIX projesinin tamamlanmasıyla birlikte farklı paydaş gruplarına yönelik çeşitli katkılar sağlanması beklenmektedir.

Paydaş	Sağlanacak Fayda
Bireysel Kullanıcılar	Oltalama saldırılarına karşı gerçek zamanlı koruma, kimlik bilgisi ve finansal veri güvenliği
Kurumsal Kullanıcılar	Çalışanların oltalama saldırılarına maruz kalma riskinin azaltılması, kurumsal veri güvenliğinin artırılması
Siber Güvenlik Araştırmacıları	Derin öğrenme tabanlı tespit yöntemlerinin pratik uygulaması için referans çalışma
Akademik Topluluk	Makine öğrenmesi ve siber güvenlik alanlarını birleştiren uygulamalı bir araştırma örneği

A.4. Yöntem

PHOENIX sistemi, birbirine entegre üç ana bileşenden oluşan katmanlı bir mimariyle gerçekleştirilecektir: (1) makine öğrenmesi tabanlı tespit modeli, (2) RESTful arka uç servisi ve (3) Google Chrome tarayıcı eklentisi. Bu üç bileşen birlikte çalışarak kullanıcının ziyaret ettiği web sayfasının oltalama amaçlı olup olmadığını gerçek zamanlı olarak değerlendirecek ve sonucu kullanıcıya anlık olarak bildirecektir.

A.4.1. Araştırma Tasarımı ve Yaklaşım

Model eğitimi için Vrbancič ve ark. (2020) tarafından hazırlanan ve kamuya açık hale getirilen veri seti kullanılacaktır. Bu veri seti; URL geneli, alan adı, dizin, dosya adı, parametre ve harici servis metrikleri olmak üzere 6 ana kategoride gruplandırılmış 111 özellik içermekte olup 58.645 ile 88.647 arasında değişen web sitesi örneğinden oluşmaktadır. Örnekler; PhishTank kaynaklı doğrulanmış oltalama URL'leri ile Alexa üst sıra sitelerinden ve topluluk etiketli meşru URL'lerden derlenmiştir.

Ön işleme aşamasında eksik değer analizi, aykırı değer tespiti ve özellik normalizasyonu uygulanacaktır. Veri seti, model eğitimi için %70 eğitim, %15 doğrulama ve %15 test olarak bölünecektir. Sınıf dengesizliği durumunda SMOTE (Synthetic Minority Oversampling Technique) yöntemi değerlendirilecektir.

A.4.2. Derin Öğrenme Modeli

Sınıflandırma görevi için öncelikli olarak tam bağlantılı (fully connected) derin sinir ağı (DNN) mimarisi kullanılacaktır. Bu temel modele ek olarak, URL karakter dizisi özelliklerini işlemek amacıyla 1D Evrişimli Sinir Ağı (CNN) ve Long Short-Term Memory (LSTM) mimarileri de kıyaslamalı olarak denenecektir.

Model performansı doğruluk (accuracy), kesinlik (precision), duyarlılık (recall) ve F1-score metrikleriyle değerlendirilecektir. Aşırı öğrenmeyi önlemek amacıyla Dropout katmanları ve L2 düzenleme uygulanacak; hiperparametre optimizasyonu için ızgara arama (grid search) yöntemi kullanılacaktır. Eğitim ortamı olarak Python, TensorFlow/Keras ve scikit-learn kütüphaneleri tercih edilecektir.

A.4.3. Backend API Geliştirme

Eğitilen model, Python tabanlı Flask veya FastAPI çerçevesi kullanılarak bir RESTful web servisi olarak yayımlanacaktır. Bu servis; Chrome eklentisinden gelen URL isteklerini alacak, gerekli özellikleri çıkaracak, modeli çalıştıracak ve tahmin sonucunu JSON formatında eklentiye geri iletecektir. Servis yerel ortamda çalışacak şekilde tasarlanacak, ilerleyen aşamada bulut tabanlı dağıtım seçenekleri değerlendirilebilecektir.

A.4.4. Chrome Tarayıcı Eklentisi

Tarayıcı eklentisi, Google Chrome Manifest V3 standardına uygun biçimde geliştirilecektir. Eklentinin temel bileşenleri şunlardır: kullanıcının ziyaret ettiği sayfanın URL bilgisini yakalayıp backend servise ileten arka plan servisi (service worker), tehdit düzeyine göre renk kodlu görsel uyarılar sunan açılır pencere (popup) ve sayfa üzerinde iletişim kuran içerik betiği (content script). Eklenti; meşru, şüpheli ve ortalama olmak üzere üç düzeyli bir değerlendirme sunacak; her düzey için kullanıcıya net ve anlaşılır bir görsel bildirim gösterilecektir.

A.5. Araştırma Olanakları

Proje kapsamında kullanılacak tüm yazılım araçları ve veri kaynakları ücretsiz ve açık kaynaklıdır; bu nedenle ek finansman veya kurumsal kaynak gerektiren bir geliştirme ortamına ihtiyaç duyulmamaktadır.

A.5.1. Donanım Olanakları

Model eğitimi ve geliştirme çalışmaları, proje üyelerinin kişisel bilgisayarları üzerinde yürütülecektir. GPU gerektiren eğitim süreçleri için Google Colab platformunun ücretsiz Tesla T4 GPU desteğinden yararlanılacaktır. Bu kaynak, söz konusu ölçekteki derin öğrenme modelleri için yeterli hesaplama kapasitesi sunmaktadır.

A.5.2. Yazılım ve Araçlar

- **Python 3.10+** — Temel geliştirme dili
- **TensorFlow / Keras** — Derin öğrenme model geliştirme
- **scikit-learn** — Veri ön işleme, model karşılaştırma ve metrik hesaplama
- **pandas, NumPy** — Veri analizi ve sayısal işlemler
- **Flask / FastAPI** — Backend REST API geliştirme
- **HTML, CSS, JavaScript** — Chrome eklentisi arayüzü
- **Git / GitHub** — Versiyon kontrolü ve işbirliği
- **Google Colab** — Bulut tabanlı GPU destekli model eğitimi
- **VS Code** — Geliştirme ortamı (IDE)

A.5.3. Veri Kaynakları

- Vrbancič ve ark. (2020) Phishing Websites Dataset — Temel eğitim veri seti (UCI ML Repository)
- PhishTank — Güncel ortalama URL'leri için referans kaynak
- Alexa Top Sites — Meşru web sitesi örnekleri

Proje danışmanı Dr. Öğr. Üyesi İlker Özçelik'in siber güvenlik alanındaki akademik birikimi ve sektör deneyimi, teknik yönlendirme açısından önemli bir araştırma olanağı olarak değerlendirilmektedir.

A.6. Çalışma Takvimi

A.6.1. İş Zaman Çizelgesi

Aşağıdaki tablo, PHOENIX projesinin 2025-2026 akademik yılı boyunca planlanmış çalışma takvimine ilişkin Gantt şemasını göstermektedir. Proje Ekim 2025'te başlayıp Mayıs 2026'daki final sunumuyla tamamlanması öngörülmektedir.

Görev / Aktivite	Şubat				Mart				Nisan				Mayıs			
	H1	H2	H3	H4	H1	H2	H3	H4	H1	H2	H3	H4	H1	H2	H3	H4
Literatür Taraması ve Araştırma																
Veri Seti Analizi ve Ön İşleme																
Özellik Çıkarımı (Feature Extraction)																
Derin Öğrenme Modeli Tasarımı																
Model Eğitimi ve Optimizasyonu																
Model Değerlendirmesi (Metrikler)																
Backend API Geliştirme																
Chrome Eklentisi Geliştirme																
Entegrasyon ve Test																
Raporlama ve Sunum Hazırlığı																

H1-H4: Haftalar (1. hafta – 4. hafta)

Yukarıdaki iş zaman çizelgesi, projenin akademik takvime uygun biçimde ilerlediğini ve kritik aşamaların dönem sonu değerlendirmelerinden önce tamamlandığını göstermektedir. Takvimde öngörülemeyen gecikmeler için her aşamaya yaklaşık iki haftalık tampon süre eklenmiştir.

A.6.2. Kim Ne İş Yapacak

Proje görev dağılımı, ekip üyelerinin güçlü yönleri ve proje gereksinimlerinin dengeli paylaşımı gözetilerek belirlenmiştir. Her üye kendi sorumluluk alanına odaklanırken literatür taraması, raporlama ve sunum gibi ortak görevler birlikte yürütülecektir.

Proje Üyesi	Sorumluluk Alanı	Detaylı Görevler
Dide Şengül	Yapay Zeka / ML Geliştirme	- Veri seti analizi ve ön işleme adımları - Özellik çıkarımı ve seçimi (111 URL özelliği) - Derin öğrenme modeli tasarımı ve eğitimi - Model performans değerlendirmesi (accuracy, F1-score, recall, precision) - Model optimizasyonu ve hiperparametre ayarı
Ata Can Özcihan	Yazılım Geliştirme / Entegrasyon	- Backend REST API tasarımı ve geliştirmesi (Flask/FastAPI) - Google Chrome Manifest V3 eklentisi geliştirme - Model ile eklenti arasında entegrasyon - Kullanıcı arayüzü (popup, uyarı bildirimleri) tasarımı - Sistem testi, hata ayıklama ve raporlama

Her İki Üye	Ortak Sorumluluklar	- Literatür taraması ve akademik kaynak analizi - Proje raporu ve dokümantasyon yazımı - Ara sınav ve final sunumu hazırlığı
--------------------	----------------------------	--

İki üye arasındaki görev dağılımı, projenin ML katmanı ile yazılım geliştirme katmanının paralel ilerledirilmesine olanak tanımakta; bu sayede toplam geliştirme süresi optimize edilmektedir. Her iki üye de haftalık durum toplantıları aracılığıyla birbirinin ilerlemesini takip edecek ve entegrasyon sürecinde aktif işbirliği yapacaktır.

B. ANALİZ

B.1. Sistem Gereksinimlerini Ortaya Çıkarma Yöntem ve Teknikleri

B.1.1. Yüz Yüze Görüşme

PHOENIX sisteminin gereksinimlerinin belirlenmesi sürecinde, proje danışmanı Dr. Öğr. Üyesi İlker Özçelik ile gerçekleştirilen yüz yüze görüşmelerden yararlanılmıştır. Bu görüşmeler, projenin kapsamının netleştirilmesi, teknik yaklaşımın belirlenmesi ve uygulanacak yöntemlerin doğrulanması açısından önemli bir rol oynamıştır.

Görüşmeler sırasında özellikle sistemin yalnızca akademik bir model geliştirme çalışması olarak kalmaması, aynı zamanda gerçek dünya kullanımına uygun bir çözüm sunması gerektiği vurgulanmıştır. Bu doğrultuda başlangıçta planlanan “tarayıcı eklentisi” yaklaşımı gözden geçirilmiş ve sistemin öncelikli olarak ortalama tespiti yapan çekirdek bir model olarak geliştirilmesi, ardından bu modelin uygun bir arayüz ile kullanıcıya sunulması yönünde karar alınmıştır. Bu yaklaşım, sistemin modüler bir yapıda geliştirilmesine olanak sağlamıştır.

Sonuç olarak, gerçekleştirilen yüz yüze görüşmeler projenin teknik doğruluğunu artırmış, kapsamını netleştirmiş ve geliştirilecek sistemin hem akademik hem de pratik açıdan daha güçlü bir temele oturmasını sağlamıştır.

B.1.2. Yazılı Basılı Belge İnceleme

PHOENIX sisteminin gereksinimlerinin belirlenmesi sürecinde, ortalama (phishing) tespiti alanında yapılmış akademik çalışmalar, veri seti dokümanları ve mevcut güvenlik sistemlerine ait teknik kaynaklar incelenmiştir. Bu inceleme süreci, kullanılacak yöntemlerin belirlenmesi ve sistemin teknik altyapısının oluşturulması açısından önemli bir temel sağlamıştır.

İncelenen başlıca çalışmalar ve elde edilen bulgular aşağıda özetlenmiştir:

- Mohammad ve ark. (2012):** Ortalama tespitinde URL, HTML ve alan adı özelliklerinin birlikte kullanımının sınıflandırma başarımını artırdığı gösterilmiştir. Özellik seçiminin model performansı üzerinde kritik rol oynadığı vurgulanmıştır.
- Abdelhamid ve ark. (2014):** İlişkilendirme kuralı tabanlı sınıflandırma yaklaşımı önerilmiş ve geleneksel yöntemlerle rekabetçi sonuçlar elde edilmiştir.
- Vrbančić ve ark. (2019, 2020):** Derin öğrenme modellerinin phishing tespitinde yüksek performans gösterdiği ortaya konmuştur. Ayrıca 111 URL tabanlı özellik içeren veri seti geliştirilmiş olup bu veri seti bu projede temel kaynak olarak kullanılmıştır.
- Veri Seti İncelemesi:** Kullanılan veri setinde URL uzunluğu, özel karakter sayısı, alan adı yapısı ve şüpheli kelimeler gibi özelliklerin sınıflandırma üzerinde etkili olduğu belirlenmiştir.
- Mevcut Sistemler (Google Safe Browsing, VirusTotal, PhishTank):** Bu sistemlerin çoğunlukla kara liste tabanlı çalıştığı ve yeni oluşturulan phishing sitelerini tespit etmede gecikmeler yaşadığı gözlemlenmiştir.

Bu incelemeler sonucunda, PHOENIX sisteminin derin öğrenme tabanlı, URL özelliklerine dayanan ve gerçek zamanlı analiz yapabilen bir yapı ile geliştirilmesinin uygun olduğu değerlendirilmiştir. Yazılı ve basılı kaynaklardan elde edilen bilgiler, sistem gereksinimlerinin belirlenmesinde ve tasarım kararlarının şekillendirilmesinde doğrudan etkili olmuştur.

B.1.3. Anket

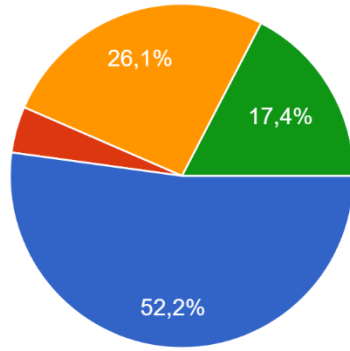
Kullanıcı gereksinimlerinin belirlenmesi amacıyla bir anket hazırlanmıştır. Bu kapsamda, kullanıcıların phishing (oltalama) saldırılarıyla ilgili deneyimlerini, karşılaştıkları tehditleri ve tarayıcı güvenlik araçlarından beklentilerini anlamak amacıyla hazırlanmıştır. Anket sonuçları, proje geliştirme sürecinde kullanıcı gereksinimlerini belirlemek, PHOENIX eklentisinin işlevselliğini optimize etmek ve sistemin gerçek kullanıcı ihtiyaçlarına uygunluğunu değerlendirmek için kullanılacaktır. Anket soruları aşağıdaki başlıklara odaklanmaktadır:

- **Phishing Saldırısı Deneyimi:** Katılımcıların daha önce phishing saldırısına maruz kalıp kalmadıkları ve bu saldırıları fark edip etmedikleri değerlendirilir.
- **Tehdit Farkındalığı:** Kullanıcıların sahte web sitelerini tespit etmek için hangi yöntemlere başvurduğu sorgulanır.
- **Güvenlik Aracına Yönelik İlgi:** Otomatik phishing tespiti yapan bir tarayıcı eklentisine olan ihtiyaç ve kullanım eğilimi belirlenir.
- **Uyarı Karşısındaki Kullanıcı Davranışı:** Kullanıcıların güvenlik uyarısı aldıklarında nasıl tepki verdikleri incelenir.
- **Algılanan Güvenlik Düzeyi:** Katılımcıların phishing tehditlerine karşı kendilerini ne ölçüde güvende hissettikleri değerlendirilir.

Soru 1: İnternette gezinirken daha önce bir phishing (oltalama) saldırısına maruz kaldınız mı?

İnternette gezinirken daha önce bir phishing (oltalama) saldırısına maruz kaldınız mı?

46 yanıt

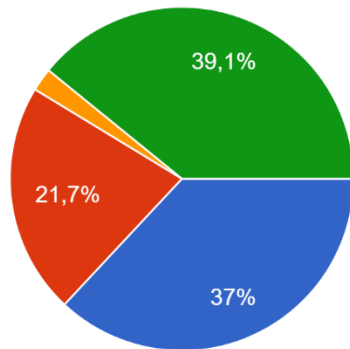


- Evet, maruz kaldım ve farkında vardım
- Evet, maruz kaldım ama sonradan fark ettim
- Hayır, hiç maruz kalmadım
- Emin değilim

Soru 2: Bir web sitesinin sahte (phishing) olup olmadığını nasıl anlarsınız?

Bir web sitesinin sahte (phishing) olup olmadığını nasıl anlarsınız?

46 yanıt

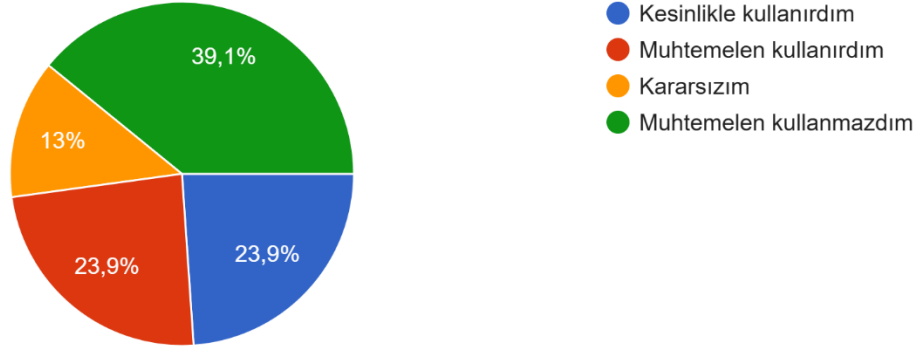


- URL'yi dikkatle incelerim
- Tarayıcının güvenlik uyarılarına güvenirim
- Sitenin görsel tasarımına bakarak karar veririm
- Genellikle fark etmem

Soru 3: Tarayıcınıza kurulu bir eklenti ziyaret ettiğiniz sitenin phishing olduğunu otomatik olarak tespit edip sizi uyarırsaydı, bu eklentiye kullanır mıydınız?

Tarayıcınıza kurulu bir eklenti ziyaret ettiğiniz sitenin phishing olduğunu otomatik olarak tespit edip sizi uyarırsaydı, bu eklentiye kullanır mıydınız?

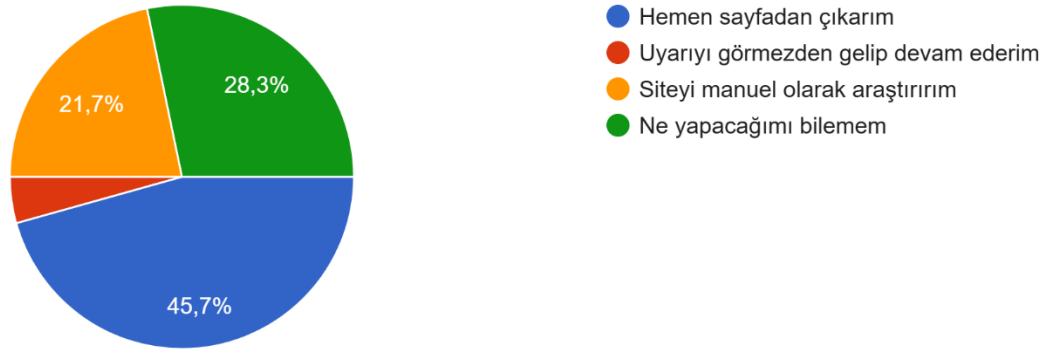
46 yanıt



Soru 4: Bir web sitesinin phishing olduğuna dair uyarı aldığınızda ne yaparsınız?

Bir web sitesinin phishing olduğuna dair uyarı aldığınızda ne yaparsınız?

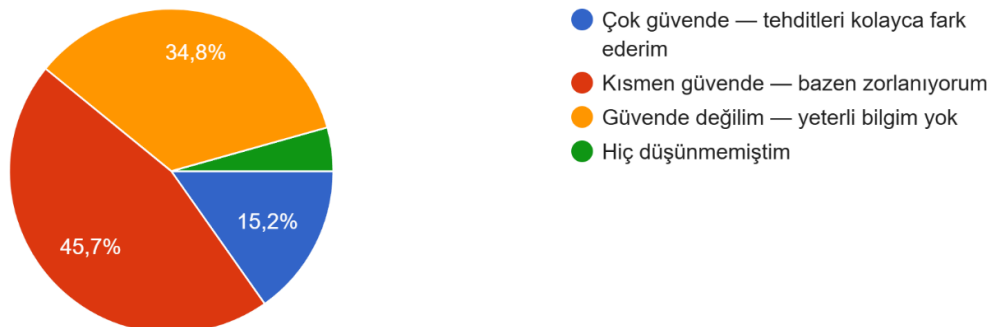
46 yanıt



Soru 5: Phishing saldırılarına karşı kendinizi ne kadar güvende hissediyorsunuz?

Phishing saldırılarına karşı kendinizi ne kadar güvende hissediyorsunuz?

46 yanıt



B.1.4. Gözlem

PHOENIX sisteminin gereksinimlerinin belirlenmesi sürecinde, kullanıcıların internet kullanım alışkanlıkları ve mevcut tarayıcı güvenlik araçlarının davranışları gözlemlenmiştir. Özellikle kullanıcıların çoğu zaman ziyaret ettikleri web sitelerinin güvenilirliğini kontrol etmeden işlem yaptığı ve tarayıcı uyarılarını dikkate almadan ilerlediği gözlemlenmiştir.

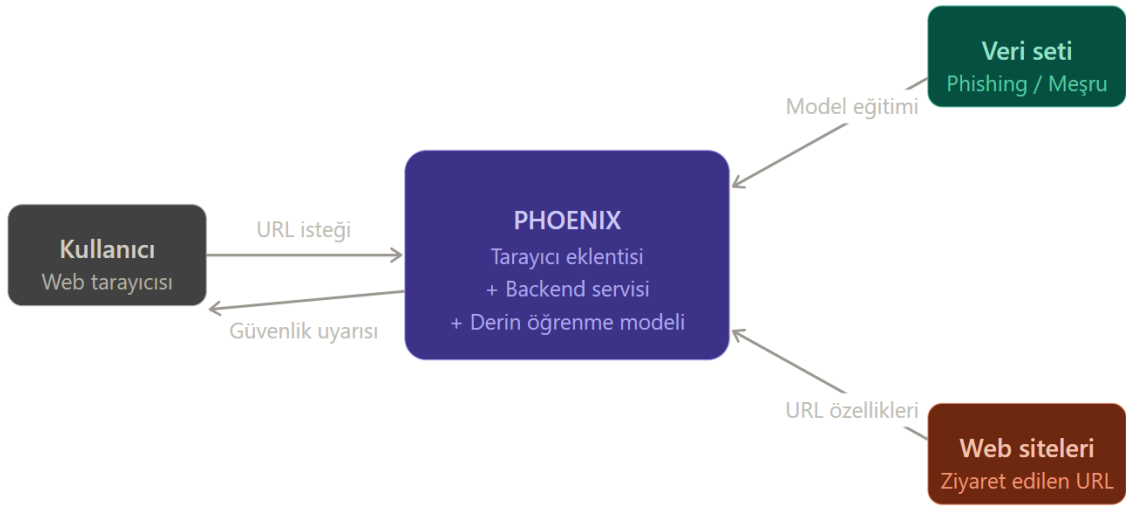
Ayrıca mevcut tarayıcı tabanlı güvenlik çözümlerinin genellikle pasif uyarılar sunduğu ve kullanıcıyı yeterince yönlendirmediği tespit edilmiştir. Bu durum, ortalama saldırıların kullanıcılar üzerinde etkili olmasını kolaylaştırmaktadır.

Yapılan gözlemler sonucunda, kullanıcıya aktif, anlaşılır ve anlık geri bildirim sağlayan bir sistemin gerekliliği ortaya konmuştur. Bu doğrultuda PHOENIX sisteminin, kullanıcı müdahalesi gerektirmeden çalışan ve doğrudan yönlendirici uyarılar sunan bir yapı ile tasarlanmasına karar verilmiştir.

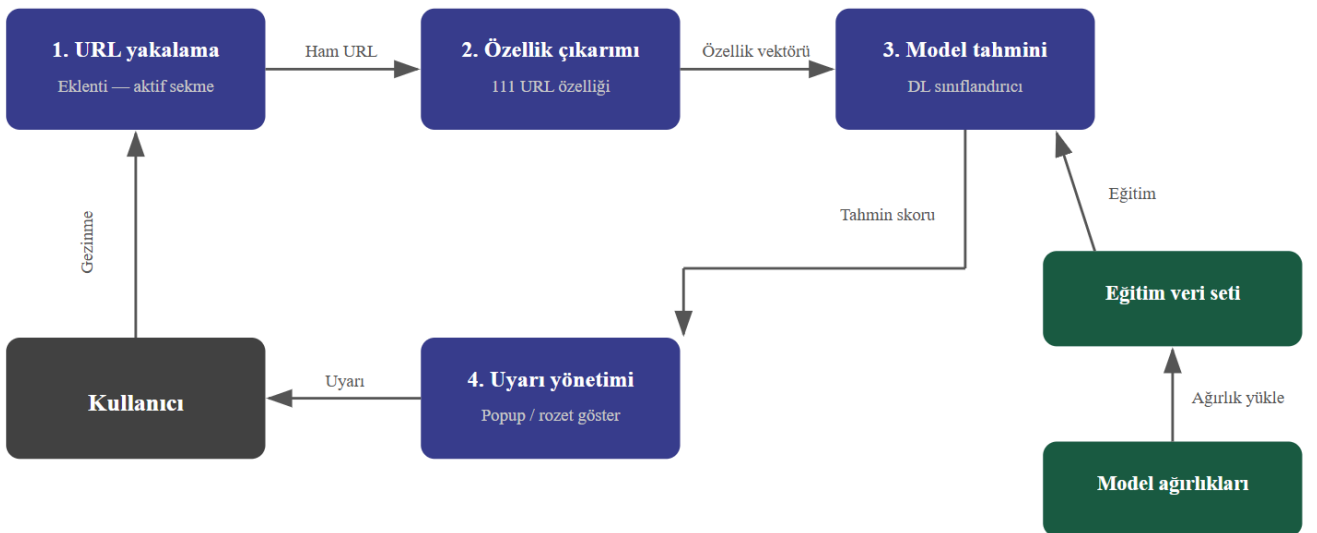
B.1.5. Veri Akış Şemaları

B.1.5.1. Kavramsal Veri Akış Şeması

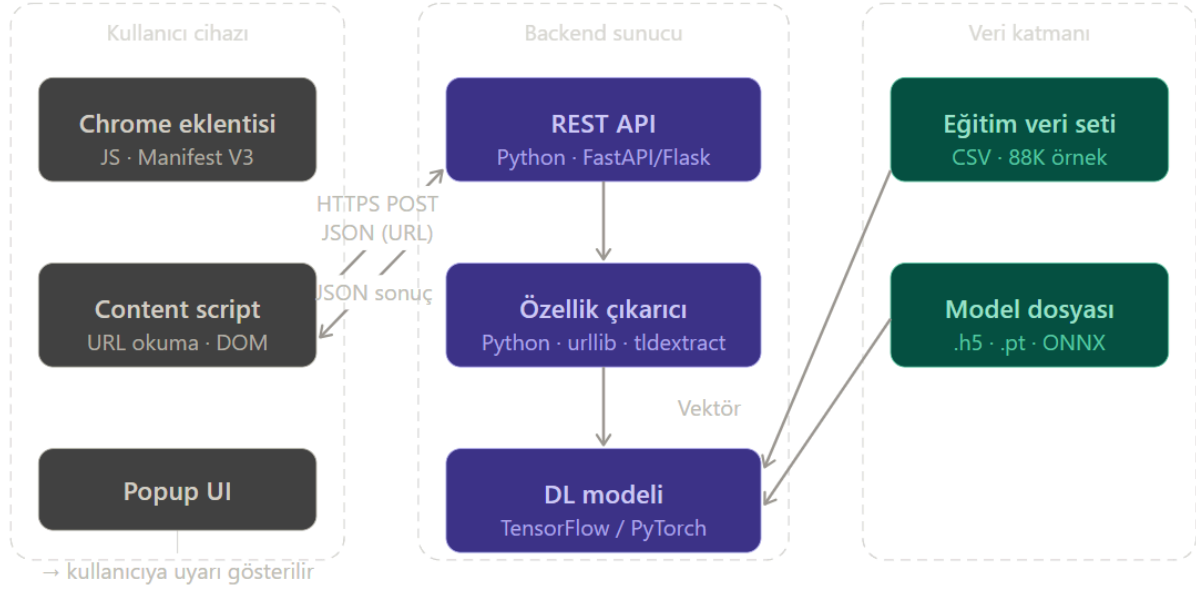
B.1.5.1 — Kavramsal veri akış şeması (Seviye U)



B.1.5.2. Mantıksal Veri Akış Şeması



B.1.5.3. Fiziksel Veri Akış Şeması



B.1.6. Olay Tabloları, Durum Formları, İşlevsel Analiz Raporu, İş Akış Şeması

Olay No	Olay adı	Tür	Tetikleyici	Sistem yanıtı	İlgili süreç
O-01	Kullanıcı yeni sekmeye gider	Dış	Tarayıcıda URL değişir	Eklenti aktif sekmeyi dinler, URL'yi yakalar	1. URL Yakalama
O-02	URL backend'e iletilir	İç	O-01 tamamlanır	Content script HTTPS POST isteği gönderir	1 → 2 geçiş
O-03	Özellik çıkarımı başlar	İç	Backend URL isteği alır	111 URL özelliği hesaplanır (uzunluk, özel karakter, subdomain vb.)	2. Özellik Çıkarımı
O-04	Model tahmini yapılır	İç	Özellik vektörü hazır	DL modeli skoru üretir (0–1 arası olasılık)	3. Model Tahmini
O-05	Phishing tespit edilir	İç	Skor ≥ eşik değeri (ör. 0.7)	Kırmızı uyarı popup'ı gösterilir, rozet güncellenir	4. Uyarı Yönetimi
O-06	Site güvenli bulunur	İç	Skor < eşik değeri	Yeşil onay rozeti gösterilir	4. Uyarı Yönetimi
O-07	Kullanıcı uyarıyı kapatır	Dış	Popup'taki "Kapat" butonuna tıklanır	Popup gizlenir, sekme açık kalır	4. Uyarı Yönetimi
O-08	Kullanıcı yine de devam eder	Dış	"Yine de devam et" seçilir	Karar loglanır, sekme açık kalır	4. Uyarı Yönetimi
O-09	Backend yanıt vermez	İç	API timeout (≥ 5 sn)	Sarı "analiz edilemedi" rozeti gösterilir	2. Özellik Çıkarımı
O-10	Model yeniden eğitilir	Zamanlı	Yeni veri seti versiyonu yüklenir	Model dosyası güncellenir, backend yeniden başlatılır	3. Model Tahmini

B.1.6.1 Durum Formları

● Bekleme durumu	D-01	● Analiz durumu	D-02
Açıklama	Eklenti yüklü, kullanıcı henüz gezinmedi	Açıklama	URL yakalandı, backend işliyor
Giriş olayı	Tarayıcı başlatılır	Giriş olayı	O-01 URL yakalama
Çıkış olayı	Kullanıcı URL'ye gider (O-01)	Çıkış olayı	Model skoru döner (O-04)
Sonraki durum	D-02 Analiz	Sonraki durum	D-03 veya D-04 veya D-05
Rozet	Gri — nötr	Rozet	Mor — analiz ediliyor
● Güvenli durum	D-03	● Tehlike durumu	D-04
Açıklama	Site meşru olarak sınıflandırıldı	Açıklama	Phishing tespit edildi, uyarı gösteriliyor
Giriş olayı	O-06 — skor < eşik	Giriş olayı	O-05 — skor ≥ eşik
Çıkış olayı	Kullanıcı yeni URL'ye gider	Çıkış olayı	O-07 kapat veya O-08 devam
Sonraki durum	D-02 Analiz	Sonraki durum	D-01 veya D-02
Rozet	Yeşil — güvenli	Rozet	Kırmızı — tehlike
● Belirsiz durum	D-05	● Model güncelleme	D-06
Açıklama	Backend yanıt vermedi veya hata oluştu	Açıklama	Yeni veri setiyle model yeniden eğitiliyor
Giriş olayı	O-09 — timeout/hata	Giriş olayı	O-10 — yeni veri seti yüklendi
Çıkış olayı	Kullanıcı yeni URL'ye gider	Çıkış olayı	Eğitim tamamlanır, model deploy edilir
Sonraki durum	D-02 Analiz	Sonraki durum	D-01 Bekleme
Rozet	Sarı — analiz edilemedi	Rozet	Mavi — güncelleniyor

B.1.6.2 İşlevsel Analiz Raporu

Temel işlevler

İşlev No	İşlev adı	Girdi	Çıktı	Öncelik
F-01	URL yakalama	Aktif sekme URL'si	Ham URL string	Zorunlu
F-02	Özellik çıkarımı	Ham URL	111 boyutlu özellik vektörü	Zorunlu
F-03	Phishing sınıflandırma	Özellik vektörü	0–1 olasılık skoru	Zorunlu
F-04	Kullanıcı uyarısı	Tahmin skoru	Görsel popup + rozet	Zorunlu
F-05	Hata yönetimi	API hatası / timeout	Sarı "belirsiz" rozeti	Zorunlu
F-06	Model eğitimi	Etiketli veri seti (CSV)	Eğitilmiş model dosyası	Zorunlu
F-07	Eşik yapılandırma	Admin parametresi	Güncellenmiş eşik değeri	Opsiyonel
F-08	Karar loglama	URL + tahmin skoru	Log kaydı (yerel / sunucu)	Opsiyonel

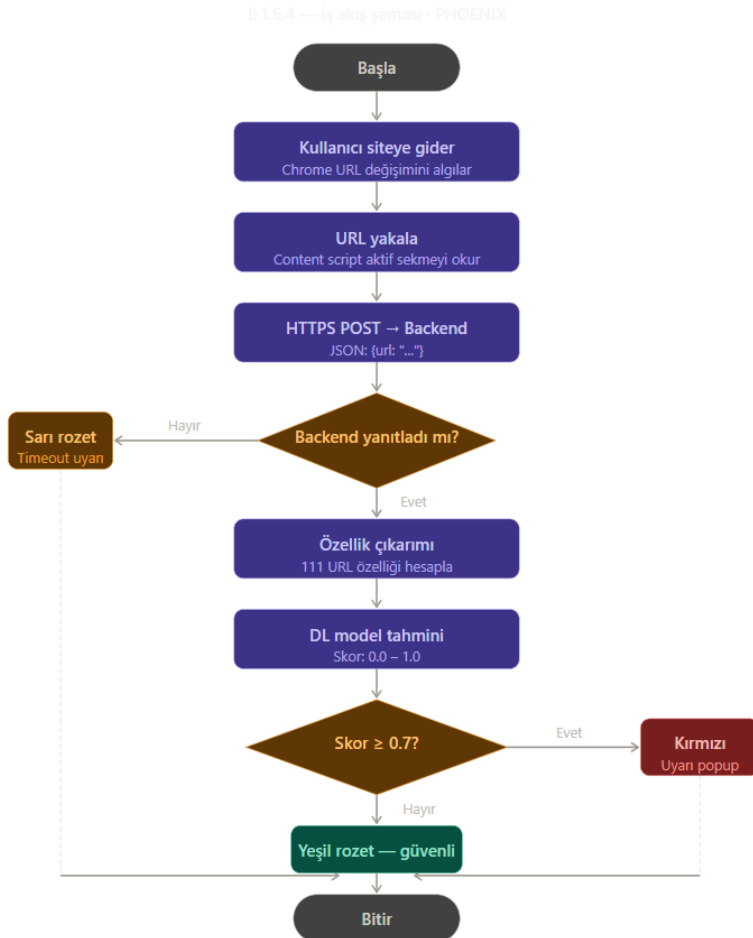
Özellik grupları(F-02 detayı)

Grup	Örnekler	Özellik sayısı
URL geneli	Uzunluk, özel karakter sayısı, @ işareti, // varlığı	~20
Alan adı	Subdomain sayısı, tire işareti, IP adresi mi?	~18
Dizin (directory)	Dizin derinliği, özel karakter	~12
Dosya adı	Uzunluk, uzantı türü	~10
Parametre	Parametre sayısı, uzunluğu	~15
Çözümleme & harici	WHOIS, Google index, PageRank, DNS kaydı	~36

Kısıtlar ve varsayımlar

Tür	Madde
Kısıt	E-posta phishing, görsel benzerlik ve HTTPS sertifika doğrulaması kapsam dışı
Kısıt	Sistem yalnızca Chrome Manifest V3 ile çalışır
Kısıt	Backend yanıt süresi < 2 saniye olmalıdır (UX gereksinimi)
Varsayım	Kullanıcının internet bağlantısı vardır
Varsayım	Vrbanič (2020) veri seti güncel phishing örüntülerini yeterince temsil eder

B.1.6.4 İş Akış Şeması



B.2. Sistem Gereksinimleri

B.2.1. İşlevsel Gereksinimler

İşlevsel gereksinimler, PHOENIX sisteminin kullanıcıya sunması gereken davranışları ve yerine getirmesi zorunlu işlevleri tanımlamaktadır. Aşağıdaki tabloda her gereksinim benzersiz bir kimlik (ID), açıklama ve öncelik düzeyiyle birlikte sunulmaktadır.

ID	Gereksinim Adı	Açıklama	Öncelik
FR-01	URL Yakalama	Sistem, kullanıcının Chrome tarayıcısında ziyaret ettiği web sitesinin tam URL bilgisini otomatik olarak yakalamalıdır. Yeni sekme açılması ve sayfa yenilenmesi durumlarında da tetiklenmelidir.	Yüksek
FR-02	Özellik Çıkarımı	Sistem, yakalanan URL'den Vrbančić (2020) veri setindeki 111 özelliği gerçek zamanlı olarak hesaplamalıdır. Özellikler; URL geneli, alan adı, dizin, dosya adı, parametre ve harici servis metrikleri olmak üzere 6 grupta çıkarılmalıdır.	Yüksek
FR-03	Phishing Tespiti	Sistem, çıkarılan 111 özelliği eğitilmiş DNN modeline göndermeli ve sınıflandırma sonucunu üretmelidir. Model yalnızca URL tabanlı özellikler kullanarak tahmin yapabilmelidir.	Yüksek
FR-04	Üç Düzeyli Sınıflandırma	Sistem tahmin sonucunu üç kategoride sunmalıdır: Güvenli (confidence ≥ 0.85), Şüpheli ($0.50 \leq \text{confidence} < 0.85$), Phishing (confidence ≥ 0.85 phishing yönünde). Her kategori için ayrı görsel bildirim gösterilmelidir.	Yüksek
FR-05	Güven Skoru	Sistem, her tahminle birlikte 0-100 aralığında bir güven skoru üretmeli ve kullanıcıya göstermelidir.	Orta
FR-06	Görsel Uyarı	Chrome eklentisi tehdit düzeyine göre renk kodlu uyarı göstermelidir: Güvenli → yeşil, Şüpheli → sarı, Phishing → kırmızı. Eklenti ikonu da aynı renk kodlamasıyla güncellenmelidir.	Yüksek
FR-07	Yanıt Süresi	Backend API, URL başına en fazla 2 saniye içinde yanıt vermelidir. Ağ gecikmesi hariç, özellik çıkarımı ve model tahmini toplam 1 saniyeyi aşmamalıdır.	Yüksek
FR-08	Tetikleyen Özellikler	Sistem, phishing veya şüpheli kararı verdiğinde hangi özelliklerin bu kararı tetiklediğini (en az 3 özellik) kullanıcıya göstermelidir.	Orta
FR-09	Manuel Analiz	Kullanıcı, popup arayüzü üzerinden manuel URL girerek analiz yaptırabilmelidir. Bu özellik otomatik analizden bağımsız çalışmalıdır.	Orta
FR-10	Hata Yönetimi	API'ye erişilemeyen durumlarda eklenti kullanıcıya bağlantı hatası bildirmeli, analiz yapılamadığını açıkça belirtmelidir.	Yüksek

Yukarıdaki tabloda yer alan işlevsel gereksinimler, sistemin temel davranışını belirlemekte olup C bölümündeki bileşen tasarımlarıyla (Chrome eklentisi, Flask API, DNN modeli) birebir eşleşmektedir. Özellikle FR-02 (özellik çıkarımı) ve FR-03 (phishing tespiti) gereksinimleri, sistemin çekirdek işlevini oluşturmaktadır.

B.2.2. Sistem ve Kullanıcı Arayüzleri ile ilgili Gereksinimler

PHOENIX sistemi üç farklı arayüz katmanından oluşmaktadır: Chrome eklentisi kullanıcı arayüzü, backend API arayüzü ve sistem bileşenleri arasındaki dahili iletişim arayüzü.

Chrome Eklentisi Kullanıcı Arayüzü:

- Eklenti simgesi (badge), aktif sekmedeki analiz sonucuna göre otomatik olarak renk değiştirmelidir: yeşil (güvenli), sarı (şüpheli), kırmızı (phishing). Simge üzerinde güven skoru sayısal olarak gösterilmelidir.
- Popup penceresi maksimum 400×300 piksel boyutunda olmalı; ziyaret edilen site adı, tehdit kategorisi, güven skoru yüzdesi ve kararı tetikleyen en az 3 özellik adı tek bakışta okunabilir biçimde sunulmalıdır.
- Kullanıcı herhangi bir yapılandırma veya hesap oluşturma işlemi yapmadan eklentiye kullanabilmelidir. Kurulum Chrome Web Store üzerinden tek tıkla gerçekleşmelidir.
- Arayüz dili Türkçe olmalıdır. Teknik terim içeren ifadeler kullanıcı dostu eşdeğerleriyle gösterilmelidir (örn. "confidence score" yerine "Güvenlik Skoru: %94").
- Phishing tespiti durumunda popup, kullanıcıyı bu sayfadan uzak durması konusunda açık ve anlaşılır biçimde uyarmalı; sayfaya devam etme veya geri dönme seçeneklerini sunmalıdır.

Backend API Arayüzü:

- API, POST /predict endpoint'i üzerinden URL içeren JSON isteklerini kabul etmelidir. İstek formatı: { "url": "<string>" }.
- Başarılı yanıt formatı: { "result": "safe | suspicious | phishing", "confidence": <float 0-1>, "risk_score": <int 0-100>, "triggered_features": [<string listesi>] }.
- Hatalı veya eksik URL formatı için HTTP 400 Bad Request, API erişim hatası için HTTP 503 Service Unavailable yanıtı dönülmelidir.
- API yalnızca localhost:5000 üzerinden erişilebilir olmalı; CORS politikası yalnızca eklentinin kaynak adresine izin verecek şekilde kısıtlanmalıdır.

Sistem İçi İletişim Arayüzü:

- Chrome eklentisinin content_script.js modülü ile background.js modülü arasındaki mesajlaşma Chrome Messaging API kullanılarak gerçekleştirilmelidir.
- background.js ile Flask API arasındaki iletişim HTTP POST üzerinden JSON formatında yürütülmelidir. Bağlantı zaman aşımı 5 saniye olarak ayarlanmalıdır.

B.2.3. Veriyle İlgili Gereksinimler

PHOENIX sisteminin doğru çalışması, eğitim verisi kalitesine ve çalışma zamanı veri tutarlılığına doğrudan bağlıdır. Aşağıdaki tablo sistemin kullandığı veri türlerini, formatlarını ve kaynaklarını özetlemektedir.

Veri Türü	Format	Açıklama	Kaynak
Eğitim Veri Seti	<i>Vrbančič (2020)</i>	88.647 URL örneği, 111 özellik, ikili etiket (phishing/meşru)	UCI ML Repository
Eğitim Bölünmesi	<i>Statik bölünme</i>	%70 eğitim / %15 doğrulama / %15 test (rastgele seed=42)	Geliştirme ortamı
Özellik Vektörü	<i>Gerçek zamanlı</i>	URL başına 111 sayısal özellik, float32 formatında	Feature extractor modülü
Normalizasyon	<i>StandardScaler</i>	Eğitim setinden hesaplanan ortalama/std, çalışma zamanında aynı scaler uygulanır	scikit-learn
Model Dosyası	<i>model.h5</i>	Keras formatında kaydedilen eğitilmiş DNN ağırlıkları	TensorFlow/Keras
Model Metadata	<i>model_info.json</i>	Model versiyonu, eğitim tarihi, accuracy, F1-score, eşik değerleri	Geliştirme ortamı
Kullanıcı Verisi	<i>Saklanmaz</i>	Analiz edilen URL'ler loglanmaz, üçüncü tarafa iletilmez, oturum sonrası silinir	Gizlilik politikası

Özellik Çıkarımı Gereksinimleri:

- URL Düzeyi Özellikler: URL uzunluğu, nokta sayısı, tire sayısı, alt çizgi sayısı, özel karakter sayısı (@, ?, =, &), rakam oranı, harf oranı hesaplanmalıdır.
- Alan Adı Özellikleri: Alan adı uzunluğu, alt domain sayısı, IP adresi içerip içermediği (0/1), sayısal karakter oranı, alan adı yaşı (WHOIS sorgusu ile) hesaplanmalıdır.
- Şüpheli Kelime Tespiti: "login", "verify", "secure", "account", "update", "banking", "paypal", "confirm" gibi phishing URL'lerinde sık görülen kelimelerin varlığı kontrol edilmelidir.
- Protokol ve Yönlendirme: HTTPS kullanımı (0/1), HTTP'den HTTPS'e yönlendirme varlığı, URL içinde başka URL bulunması (0/1) kontrol edilmelidir.
- Çıkarılan tüm özellikler eğitim aşamasında kullanılan StandardScaler ile normalize edilmeli; scaler parametreleri scaler.pkl dosyasında saklanmalı ve çalışma zamanında yeniden yüklenerek uygulanmalıdır.

B.2.4. Kullanıcılar ve İnsan Faktörü Gereksinimleri, Güvenlik Gereksinimleri

B.2.4.1. Kullanıcılar ve İnsan Faktörü Gereksinimleri

- Birincil hedef kullanıcı: teknik bilgisi olmayan, günlük internet kullanan bireysel kullanıcılar. Sistem bu kullanıcı grubunun herhangi bir eğitim veya teknik bilgi gerektirmeksizin kullanabileceği şekilde tasarlanmalıdır.

- Eklenti yüklendiği anda otomatik olarak aktif hale gelmeli; kullanıcının herhangi bir ayar yapması gerekmemelidir.
- Uyarı bildirimleri kısa ve net olmalıdır. Kullanıcıyı paniğe sokmak yerine bilgilendirici bir dil kullanılmalıdır. Örnek: "Bu site güvenli görünüyor" veya "Dikkat: Bu site phishing olabilir, kişisel bilgilerinizi girmeyin."
- Yanlış pozitif (meşru siteyi phishing olarak işaretleme) durumunda kullanıcı "Bu uyarıyı yoksay" seçeneğine sahip olmalıdır; bu seçim kaydedilmemelidir.

B.2.4.2. Güvenlik Gereksinimleri

- Minimum Yetki Prensibi: Chrome eklentisi yalnızca activeTab ve scripting izinlerini talep etmelidir. Tarama geçmişi, yer işaretleri veya diğer sekme bilgilerine erişim talep edilmemelidir.
- Veri Mahremiyeti: Kullanıcının ziyaret ettiği URL'ler hiçbir harici sunucuya iletilmemelidir. API yalnızca localhost üzerinde çalışmalı; internet bağlantısı kesildiğinde sistem çevrimdışı modda uyarı göstermelidir.
- Veri Saklamama: Analiz edilen URL'ler, kullanıcı davranışları veya kişisel bilgiler hiçbir şekilde loglanmamalı, dosyaya yazılmamalı veya veri tabanında saklanmamalıdır.
- Kod Bütünlüğü: Eklenti ve API kodları GitHub'da açık kaynak olarak yayımlanmalı; bağımlılıklar sabitlenmiş versiyon numaralarıyla (requirements.txt) yönetilmelidir.

B.2.5. Teknik ve Kaynak Gereksinimleri, Fiziksel Gereksinimler

B.2.5.1. Teknik Gereksinimler

PHOENIX sisteminin geliştirilmesi ve çalıştırılması için gerekli teknik gereksinimler; yazılım, donanım ve performans ihtiyaçları kapsamında belirlenmiştir. Sistem, hem geliştirme ortamında hem de son kullanıcı tarafında sorunsuz çalışacak şekilde tasarlanmıştır.

Yazılım Gereksinimleri

Araç / Kütüphane	Durum	Kullanım Amacı
Python 3.10+	Zorunlu	Tüm backend, model eğitimi ve özellik çıkarımı Python ile geliştirilecektir
TensorFlow 2.x / Keras	Zorunlu	DNN modelinin tasarımı, eğitimi ve .h5 formatında kaydedilmesi
scikit-learn 1.x	Zorunlu	Veri ön işleme, StandardScaler, metrik hesaplama (F1, precision, recall)
Flask 3.x	Zorunlu	RESTful backend API geliştirme, /predict endpoint
flask-cors	Zorunlu	Chrome eklentisinden gelen cross-origin isteklerin yönetimi
pandas / NumPy	Zorunlu	Veri seti yükleme, özellik manipülasyonu, sayısal işlemler
Google Chrome 110+	Zorunlu	Manifest V3 desteği için minimum Chrome sürümü
Google Colab (T4 GPU)	Önerilen	Model eğitimi için GPU hızlandırma, ücretsiz erişim
Git / GitHub	Zorunlu	Versiyon kontrolü, ekip işbirliği, kod yönetimi
VS Code	Önerilen	Geliştirme ortamı, Python ve JavaScript desteği
imbalanced-learn	Koşullu	Sınıf dengesizliği durumunda SMOTE uygulaması için

Donanım Gereksinimleri:

- Geliştirme Ortamı: Minimum 8 GB RAM, çift çekirdekli işlemci. Model eğitimi için GPU gereklidir; Google Colab üzerindeki ücretsiz Tesla T4 GPU bu gereksinimi karşılamaktadır.
- Çalışma Zamanı (Son Kullanıcı): Minimum 4 GB RAM, herhangi bir modern işlemci (2015 sonrası). Flask API'nin arka planda çalışması için yaklaşık 200 MB RAM tüketimi öngörülmektedir.
- Depolama: Model dosyası (.h5) ve scaler dosyası (.pkl) toplam yaklaşık 50 MB disk alanı kaplamaktadır.

Performans Gereksinimleri:

- Model doğruluğu (accuracy) $\geq$ %93 hedeflenmektedir.
- F1-score $\geq$ 0.92 hedeflenmektedir. Bu metrik, hem precision hem recall'u dengeli biçimde yansıttığından phishing tespiti gibi asimetrik sınıflandırma problemleri için temel performans göstergesi olarak benimsenmiştir.
- False Negative Oranı (phishing sitesini kaçırma) $\leq$ %5 olmalıdır. Bu, güvenlik odaklı sistemlerde false positive'e kıyasla çok daha kritik bir hata türüdür.
- API yanıt süresi (özellik çıkarımı + model tahmini) $\leq$ 2 saniye olmalıdır. Ağ gecikmesi bu sürenin dışındadır.

B.2.5.2. Fiziksel Gereksinimler:

- Sistem yalnızca Google Chrome tarayıcısı yüklü olan ve internet bağlantısına sahip bir masaüstü veya dizüstü bilgisayarda çalışacak şekilde tasarlanmaktadır.
- Mobil cihazlar ve Chrome dışındaki tarayıcılar bu projenin kapsamı dışındadır.
- Flask API ve Chrome eklentisi aynı yerel makinede (localhost) çalışmalıdır. Farklı cihazlar arasında dağıtık çalışma bu aşamada desteklenmemektedir.

C. TASARIM

C.1. Sistem Tasarımı

PHOENIX sistemi, birbirine gevşek bağlı (loosely coupled) üç ana katmandan oluşan katmanlı bir mimari üzerine inşa edilmiştir. Bu mimari yaklaşım, bileşenlerin birbirinden bağımsız geliştirilebilmesine, test edilebilmesine ve ileride genişletilebilmesine olanak tanımaktadır.

Katman 1 — Sunum Katmanı (Chrome Eklentisi): Kullanıcı ile doğrudan etkileşime giren katmandır. content_script.js modülü aktif sekmenin URL bilgisini yakalar; background.js modülü bu bilgiyi API katmanına iletir; popup.html modülü ise analiz sonucunu görsel olarak sunar. Bu katman yalnızca kullanıcı arayüzünden ve API çağrılarından sorumludur; iş mantığı içermez.

Katman 2 — Uygulama Katmanı (Flask API): Sistemin iş mantığının yürütüldüğü katmandır. Gelen URL isteklerini alır, özellik çıkarımı gerçekleştirir, modele iletir ve sonucu JSON formatında döndürür. Aynı zamanda SQLite veri tabanı ile iletişimi yönetir.

Katman 3 — Veri Katmanı (Model + SQLite): Eğitilmiş DNN modeli (.h5) ve analiz geçmişini saklayan SQLite veri tabanından oluşur. Bu katman doğrudan dış dünyayla iletişim kurmaz; yalnızca uygulama katmanı tarafından erişilir.

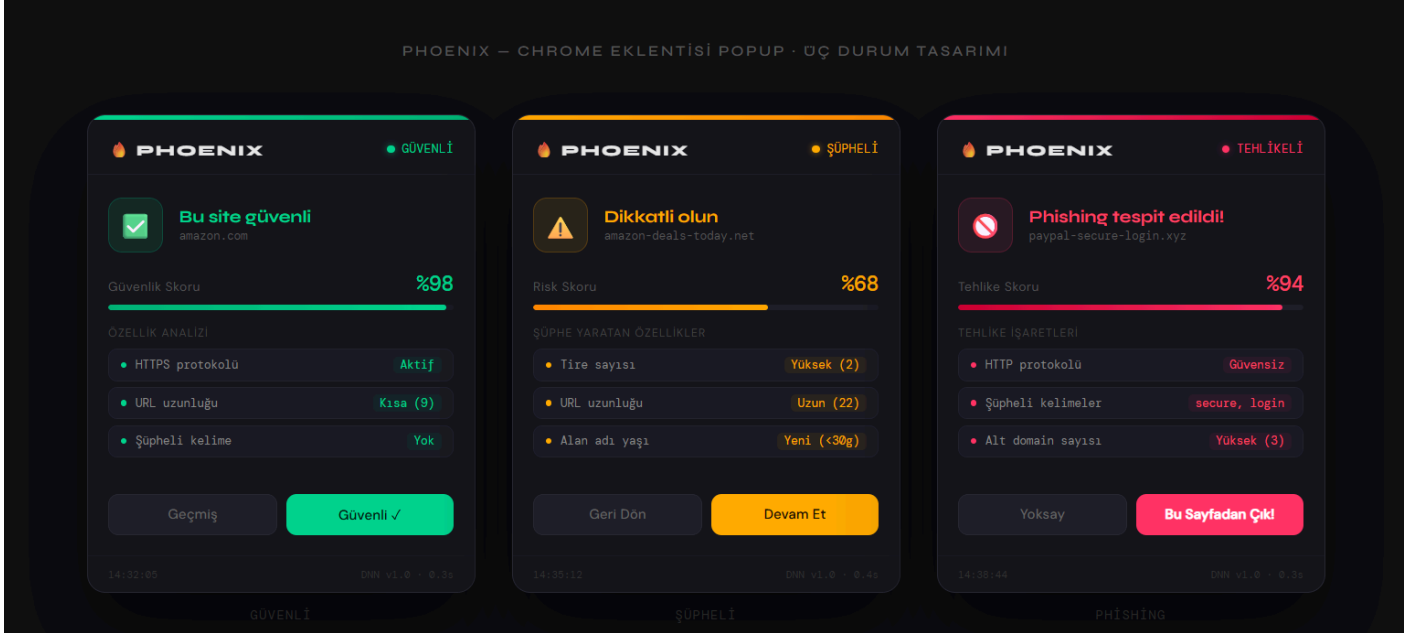
Sistem bileşenleri arasındaki iletişim tek yönlü bir akışla gerçekleşir: Chrome eklentisi → Flask API → Model/Veri tabanı → Flask API → Chrome eklentisi. Bu akış, bileşenler arasındaki bağımlılığı minimize ederek sistemin test edilebilirliğini artırmaktadır.

Tasarım Kararı	Açıklama
Mimari Stil	Katmanlı mimari (Layered Architecture) — sunum, uygulama ve veri katmanları birbirinden izole
İletişim Protokolü	HTTP/JSON — eklenti ile API arasında; Chrome Messaging API — eklenti bileşenleri arasında
Model Yönetimi	Singleton deseni — Flask başlatıldığında model bir kez yüklenir, tüm isteklerde aynı nesne kullanılır
Veri Saklama	SQLite (dosya tabanlı) — kurulum gerektirmez, localhost'ta çalışır
Hata Yönetimi	API katmanında merkezi try/except; eklenti katmanında timeout ve bağlantı hatası yakalama

C.2. Kullanıcı ve Sistem Arayüzü Tasarımları

PHOENIX sisteminin kullanıcıya yönelik arayüzleri iki katmandan oluşmaktadır: Chrome eklentisi popup penceresi ve eklentinin açılan ayarlar/dashboard sayfası. Bu iki arayüz birlikte çalışarak kullanıcıya hem otomatik analiz sonucu hem de gelişmiş özelleştirme imkânı sunmaktadır.

C.2.1. Chrome Eklentisi Popup Arayüzü



Kullanıcı Chrome tarayıcısında herhangi bir siteye girdiğinde, PHOENIX eklentisi arka planda otomatik olarak URL'yi analiz eder ve sonucu eklenti simgesinde renk kodu ile gösterir. Kullanıcı eklenti simgesine tıkladığında 380×320 piksel boyutunda bir popup penceresi açılır.

Popup penceresi üç farklı durumda görüntülenir. Güvenli durumda yeşil renk teması ve onay simgesiyle "Bu site güvenli görünüyor" mesajı gösterilir. Şüpheli durumda sarı renk teması ve ünlem simgesiyle "Dikkatli olun" uyarısı sunulur; kullanıcıya "Geri Dön" ve "Devam Et" seçenekleri sunulur. Phishing durumunda kırmızı renk teması ve yasak simgesiyle "Phishing Tespit Edildi!" uyarısı gösterilir ve "Bu Sayfadan Çık!" butonu öne çıkarılır.

Her popup ekranında; ziyaret edilen sitenin URL'si, tehdit kategorisi, 0-100 arasında güven skoru (progress bar ile görsel olarak), kararı tetikleyen en az 3 URL özelliği ve aksiyon butonları yer almaktadır. Ayrıca popup'un sağ üst köşesindeki ayarlar simgesine tıklanarak ayarlar sayfasına geçiş yapılabilir.

C.2.2. Ayarlar ve Dashboard Sayfası

PHOENIX

GENEL

Ayarlar

Analiz Geçmişi

Model Metrikleri

SİSTEM

Güvenlik

Hakkında

API Aktif · localhost:5000

Ayarlar

PHOENIX eklentisini ihtiyacınıza göre özelleştirin.

Analiz Ayarları

Otomatik analiz ve eşik değerleri

Otomatik Analiz

Her siteye girildiğinde analiz otomatik başlatılır

☒

Phishing Eşik Değeri

Bu değerin üzerindeki güven skoru phishing olarak işaretlenir

%85

Şüpheli Eşik Değeri

Bu değerin üzerindeki risk skoru şüpheli olarak işaretlenir

%50

Analiz Modu

Hangi sitelerin analiz edileceğini seçin

Tüm siteler

Bildirim Ayarları

Uyarı türleri ve davranışlar

Popup Otomatik Açılsın

Phishing tespit edildiğinde popup otomatik açılır

☒

Ses Uyarısı

Tehlikeli site tespit edildiğinde ses çıkar

☐

Phishing Sayfasını Otomatik Engelle

Yüksek riskli sitelerde sayfa otomatik olarak bloke edilir

☐

Şüpheli Siteler İçin Uyarı

Şüpheli (san) kategori için de bildirim gösterilsin

☒

Gizlilik

Veri saklama ve geçmiş yönetimi

Değişiklikler kaydedilmedi

Vazgeç

Kaydet

Gizlilik

Veri saklama ve geçmiş yönetimi

Analiz Geçmişini Kaydet

Ziyaret edilen URL'ler yerel veri tabanına kaydedilsin (localhost'ta)

☒

Maksimum Geçmiş Kaydı

Bu sayının üzerinde kayıt varsa en eskiler silinir

1.000 kayıt

Tüm Geçmişini Temizle

Kaydedilmiş tüm analiz kayıtları silinir, geri alınamaz

Geçmişini Temizle

Model Bilgisi

Aktif DNN modelinin performans metrikleri

DOĞRULUK

%94.2

Accuracy

F1 SKORU

0.941

Dengeli metrik

PRECISION

%93.8

Keskinlik

RECALL

%94.5

Duyarlılık

Eğitim Veri Seti

Vrbančić (2020) — 88.647 URL · 111 özellik

v1.0

Popup'taki ayarlar simgesine tıklandığında Chrome, tam ekran boyutunda ayrı bir web sayfası olarak ayarlar/dashboard arayüzünü açar. Bu sayfa, Chrome Manifest V3 standardındaki options_page mekanizması aracılığıyla sunulmaktadır. Sayfa sol tarafta bir kenar çubuğu (sidebar) ve ana içerik alanından oluşan iki panelli bir yapıya sahiptir.

Ayarlar sayfası beş ana bölümden oluşmaktadır. Analiz Ayarları bölümünde otomatik analiz açma/kapama, phishing eşik değeri (varsayılan %85) ve şüpheli eşik değeri (varsayılan %50) kaydırılabilir slider ile ayarlanabilmekte, ayrıca analiz modu seçilebilmektedir. Bildirim Ayarları bölümünde popup otomatik açılma, ses uyardı, phishing sayfası otomatik engelleme ve şüpheli site bildirimi seçenekleri açılıp kapatılabilmektedir. Gizlilik bölümünde analiz geçmişi kaydetme tercihi, maksimum kayıt sayısı ve geçmişi temizleme işlemleri yönetilebilmektedir. Model Bilgisi bölümünde aktif DNN modelinin accuracy, F1-score, precision ve recall değerleri görsel olarak sunulmaktadır. Hakkında bölümünde ise proje ve geliştirici bilgilerine yer verilmektedir.

C.2.3. Manuel URL Analizi

The screenshot displays the PHOENIX Phishing Detection System interface. At the top, the header includes the PHOENIX logo and navigation links: Analiz, Geçmiş, Model Metrikleri, and API Aktif. The main heading is "URL'nizi güvende tutun" (Keep your URL safe), with a subtitle: "Derin öğrenme tabanlı PHOENIX sistemi, ziyaret ettiğiniz siteleri gerçek zamanlı olarak analiz eder." (The PHOENIX system, based on deep learning, analyzes the sites you visit in real-time).

The central section shows a "URL ANALIZI" (URL Analysis) input field containing the URL "https://paypal-secure-login.xyz/verify" and an "ANALİZ ET" (ANALYZE) button. Below this, a "Phishing Tespit Edildi!" (Phishing Detected!) alert is shown for the URL "paypal-secure-login.xyz/verify". A "Tehlike Skoru" (Risk Score) bar indicates a score of %94. Factors contributing to the detection are listed: "HTTP protokolü" (HTTP protocol), "Şüpheli kelimeler: secure, login" (Suspicious words: secure, login), "Alt domain sayısı yüksek" (High number of subdomains), and "Yeni alan adı" (New domain).

Below the alert, four statistics are displayed: "MODEL DOĞRULUĞU" (Model Accuracy) at %94.2, "F1 SKORU" (F1 Score) at 0.941, "TOPLAM ANALİZ" (Total Analysis) at 1,247, and "TESPİT EDİLEN" (Detected) at 38.

The bottom section, "SON ANALİZLER" (Recent Analyses), shows a table of recent analyses:

URL	SONUÇ	GÜVEN	ZAMAN
paypal-secure-login.xyz/verify	Phishing	%94	14:38:44
amazon.com/orders	Güvenli	%98	14:35:12

Ayarlar sayfası aynı zamanda manuel URL analizi imkânı sunmaktadır. Kullanıcı, herhangi bir siteye girmeksizin URL metnini doğrudan giriş alanına yazıp "Analiz Et" butonuna basarak sorgu başlatabilir. Sistem bu URL'yi otomatik analizle aynı işlem akışından geçirir: Flask API'ye gönderir, 111 özellik çıkarılır, DNN modeli tahmin üretir ve sonuç ekranda gösterilir. Bu özellik özellikle bir bağlantıya tıklamadan önce güvenliğini kontrol etmek isteyen kullanıcılar için tasarlanmıştır. Analiz sonucu; tehdit kategorisi, güven skoru ve tetikleyen özellikler olarak aynı standart formatta sunulur.

Ayarlar sayfasının altında analiz geçmişi tablosu da yer almaktadır. Bu tablo SQLite veri tabanından çekilen son analizleri; URL, sonuç, güven skoru ve zaman bilgisiyile listeler. Böylece kullanıcı geçmişte analiz ettiği sitelerin kayıtlarına buradan erişebilir.

C.3. Test Tasarımı

C.3.1. Gereksinim Analizlerinden Test Hedeflerinin Belirlenmesi

B.2.1 bölümünde tanımlanan işlevsel gereksinimler, test hedeflerinin belirlenmesinde doğrudan temel alınmıştır. Her işlevsel gereksinim için en az bir test senaryosu oluşturulmuş; yüksek öncelikli gereksinimler için birden fazla senaryo (pozitif ve negatif) tasarlanmıştır.

Gereksinim	Test Hedefi
FR-01 URL Yakalama	Farklı site türlerinde (HTTP, HTTPS, yönlendirmeli) URL'nin doğru yakalandığı doğrulanacak
FR-02 Özellik Çıkarımı	Her URL için tam 111 özelliğin çıkarıldığı ve hiçbirinin null olmadığı kontrol edilecek
FR-03 Phishing Tespiti	1000 URL'lik test seti ile model doğruluğu ölçülecek; hedef: accuracy $\geq$ %93, F1 $\geq$ 0.92
FR-04 Üç Düzeyli Sınıflandırma	Güven eşik değerlerinin (0.50 ve 0.85) doğru uygulandığı sınır değer testleriyle doğrulanacak
FR-06 Renk Kodlu Uyarı	Her üç sonuç türü için eklenti simgesi ve popup rengi manuel olarak doğrulanacak
FR-07 Yanıt Süresi	100 ardışık istek ile ortalama yanıt süresi ölçülecek; hedef: $\leq$ 2000ms
FR-10 Hata Yönetimi	API kapalıyken, geçersiz URL girildiğinde ve ağ bağlantısı kesildiğinde sistem davranışı test edilecek

C.3.2. Fonksiyonel Test Tasarımı

Aşağıdaki tablo, PHOENIX sisteminin fonksiyonel test senaryolarını içermektedir. Her test senaryosu ilgili gereksinime referans vermekte, test adımlarını ve beklenen sonuçları açıkça tanımlamaktadır.

TC ID	Gerek sinim	Test Adı	Açıklama	Test Adımları	Beklenen Sonuç	Öncelik
TC-01	FR-01	URL Yakalama	Kullanıcı yeni bir sekme açıp bir URL'e gittiğinde eklenti bu URL'i yakalamalıdır.	1. Chrome'u aç, 2. amazon.com adresine git, 3. Popup'ı aç	Popup'ta 'amazon.com' URL'i gösterilir	Yüksek
TC-02	FR-03	Phishing Tespiti Pozitif	Bilinen phishing URL'i girildiğinde sistem 'phishing' sonucu döndürmelidir.	POST /predict {url: 'paypal-secure-login. xyz'}	result: phishing, confidence $\geq$ 0.85	Yüksek
TC-03	FR-03	Phishing Tespiti Negatif	Meşru URL girildiğinde sistem 'safe' sonucu döndürmelidir.	POST /predict {url: 'amazon.com'}	result: safe, confidence $\geq$ 0.85	Yüksek
TC-04	FR-04	Üç Düzeyli Sınıflandırma	Sistem güven skoru 0.50-0.85 arasında olan URL için 'suspicious' döndürmelidir.	POST /predict {url: şüpheli URL}	result: suspicious, 0.50 $\leq$ confidence < 0.85	Orta
TC-05	FR-06	Renk Kodlu Uyarı	Phishing sonucunda eklenti simgesi kırmızı, güvenli sonucunda yeşil olmalıdır.	TC-02 ve TC-03 sonrasında ikon rengini kontrol et	Kırmızı → phishing, Yeşil → safe	Yüksek
TC-06	FR-07	Yanıt Süresi	API yanıt süresi 2	100 farklı URL için	Ortalama yanıt	Yüksek

			saniyeyi aşmamalıdır.	/predict çağır, süreyi ölç	≤ 2000ms	
TC-07	FR-08	Tetikleyen Özellikler	Phishing kararında popup en az 3 tetikleyen özelliği göstermelidir.	Phishing URL gir, popup'ı aç	En az 3 özellik adı görünür	Orta
TC-08	FR-09	Manuel URL Girişi	Kullanıcı popup üzerinden manuel URL girerek analiz yaptırabilmelidir.	Popup'ı aç, URL gir, analiz et	Sonuç doğru şekilde gösterilir	Orta
TC-09	FR-10	Hata Yönetimi	API kapalıyken eklenti kullanıcıya bağlantı hatası bildirmelidir.	Flask API'yi durdur, bir siteye git	Popup'ta bağlantı hatası mesajı görünür	Yüksek
TC-10	FR-02	111 Özellik Çıkarımı	API her URL için tam 111 özellik üretmelidir.	POST /predict ile debug modu aktif et	features listesi 111 eleman içerir	Yüksek

Fonksiyonel testler iki aşamada yürütülecektir: birim testleri (her bileşen izole olarak) ve entegrasyon testleri (eklenti + API + model birlikte). Python'da pytest, JavaScript'te Chrome Extension test araçları kullanılacaktır.

C.3.3. Performans Test Tasarımı

Model Doğruluk Testi: Vrbančić (2020) veri setinin %15'lik test bölümü (yaklaşık 13.000 URL) kullanılarak model doğruluğu ölçülecektir. Hedef metrikler: accuracy ≥ %93, F1-score ≥ 0.92, false negative oranı ≤ %5.

API Yanıt Süresi Testi: 100 farklı URL için ardışık POST /predict istekleri gönderilecek ve her isteğin yanıt süresi kaydedilecektir. Ortalama yanıt süresinin 2000ms altında kalması beklenmektedir. Test aracı: Python requests kütüphanesi ile basit zaman ölçümü.

Eş Zamanlı İstek Testi: 5 eş zamanlı POST /predict isteği gönderilerek API'nin paralel istekleri doğru şekilde yanıtlayıp yanıtlamadığı kontrol edilecektir.

Bellek Kullanımı Testi: Flask API çalışırken sistem bellek kullanımı izlenecek; 1 saatlik sürekli kullanım sonrasında bellek sızıntısı olmadığı doğrulanacaktır.

C.4. Yazılım Tasarımı

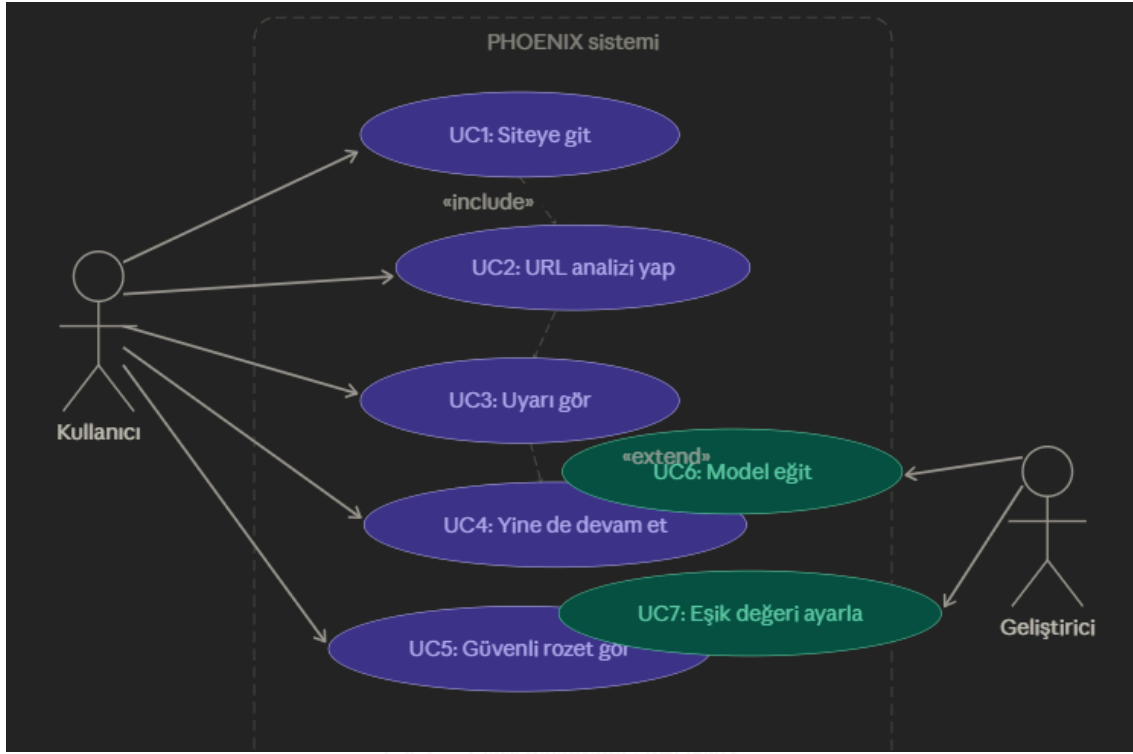
C.4.1. Gereksinime Bağlı Tasarım Kalıpları Seçimi

PHOENIX sisteminde kullanılan tasarım kalıpları, B bölümünde tanımlanan gereksinimler doğrultusunda seçilmiştir.

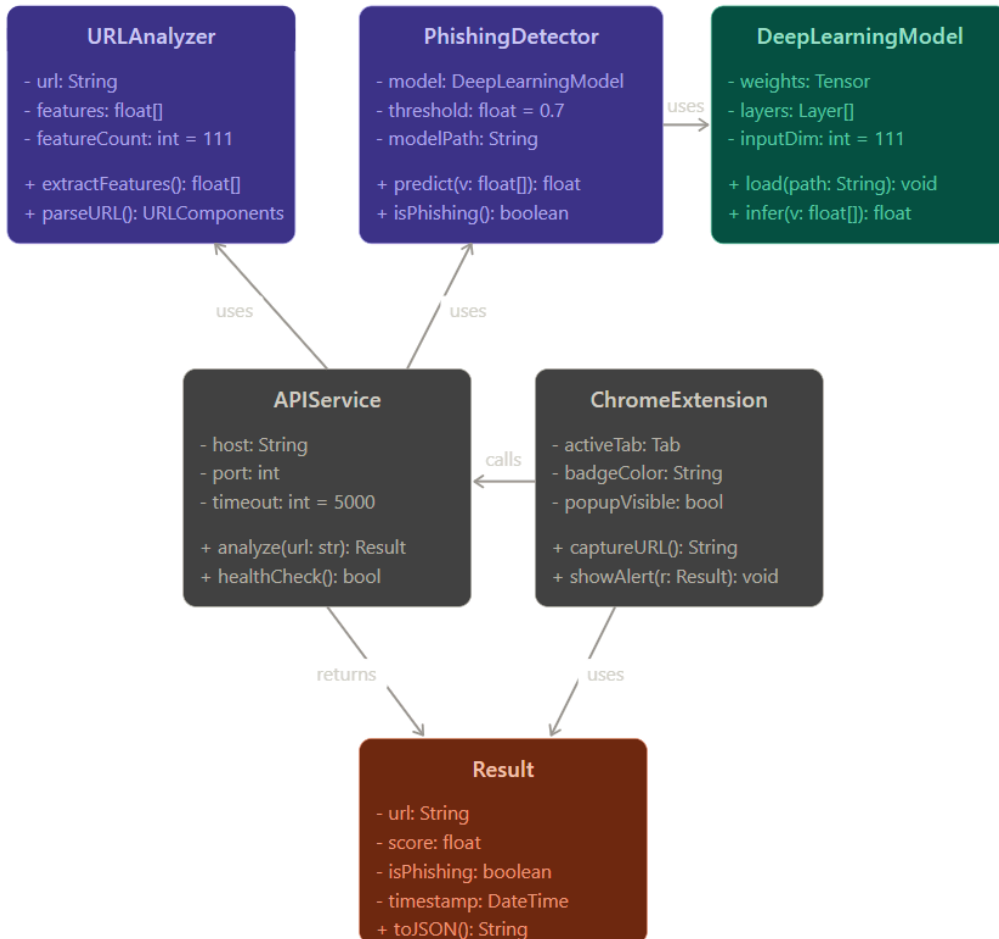
Tasarım Kalıbı	Gereksinim ve Uygulama Gerekçesi
Singleton (Model Yönetimi)	FR-07 yanıt süresi gereksinimi: DNN modeli Flask başlatıldığında bir kez belleğe yüklenir. Her istek yeni bir model yüklemeyi; aynı nesne yeniden kullanılır. Bu yaklaşım yanıt süresini önemli ölçüde azaltır.
Strategy (Özellik Çıkarımı)	FR-02 özellik çıkarımı gereksinimi: 111 özellik altı gruba ayrılmış; her grup için ayrı bir çıkarım fonksiyonu tasarlanmıştır. Yeni özellik grupları mevcut kodu değiştirmeden eklenebilir.
Observer (Sekme Takibi)	FR-01 URL yakalama gereksinimi: Chrome'un tabs.onUpdated eventi dinlenerek her sekme değişikliğinde analiz tetiklenir. Eklenti bileşenleri birbirinden bağımsız çalışır.
Repository (Veri Erişimi)	C.5 veri tabanı gereksinimi: SQLite işlemleri ayrı bir db_manager.py modülünde toplanmıştır. API endpoint'leri doğrudan SQL yazmaz; veri erişimi bu katman üzerinden gerçekleşir.
Facade (API Tasarımı)	FR-03 tespit gereksinimi: /predict endpoint'i karmaşık iç işlemleri (özellik çıkarımı, normalizasyon, model çağırısı, veri tabanı kaydı) tek bir basit arayüz arkasında gizler.

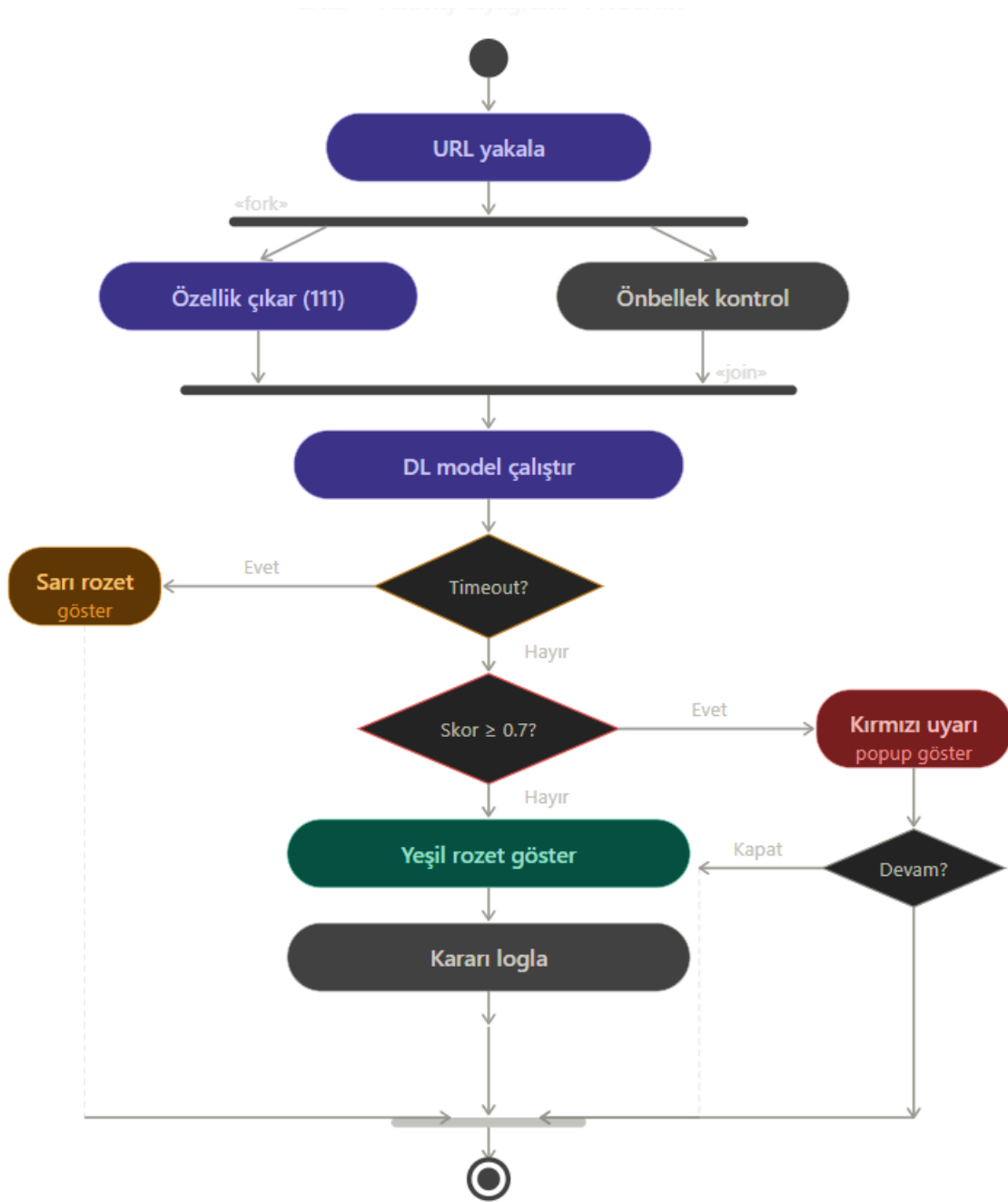
C.4.2. UML Kullanılarak Tasarım Diyagramları

Bileşen Diyagramı (Component Diagram): Sistem üç ana bileşenden oluşmaktadır. Chrome Eklentisi bileşeni; content_script, background ve popup alt bileşenlerini içerir. Flask API bileşeni; predict_service, feature_extractor ve db_manager alt bileşenlerini barındırır. Model & Veri katmanı; model.h5, scaler.pkl ve analysis.db dosyalarından oluşur.



C.4.2 – Class Diagram – PHOENIX





C.5. Veri Tabanı Tasarımı

PHOENIX sistemi, analiz geçmişini saklamak amacıyla dosya tabanlı SQLite veri tabanı kullanmaktadır. SQLite, kurulum gerektirmeyen yapısı ve Python'un standart kütüphanesinde yer alması nedeniyle bu proje için en uygun veri tabanı çözümü olarak seçilmiştir.

C.5.1. Veri Tabanı Tabloları Dökümü

Sistem tek bir tablo içermektedir: analysis_history. Bu tablo, kullanıcının Chrome eklentisi aracılığıyla gerçekleştirdiği her URL analizinin kaydını tutar.

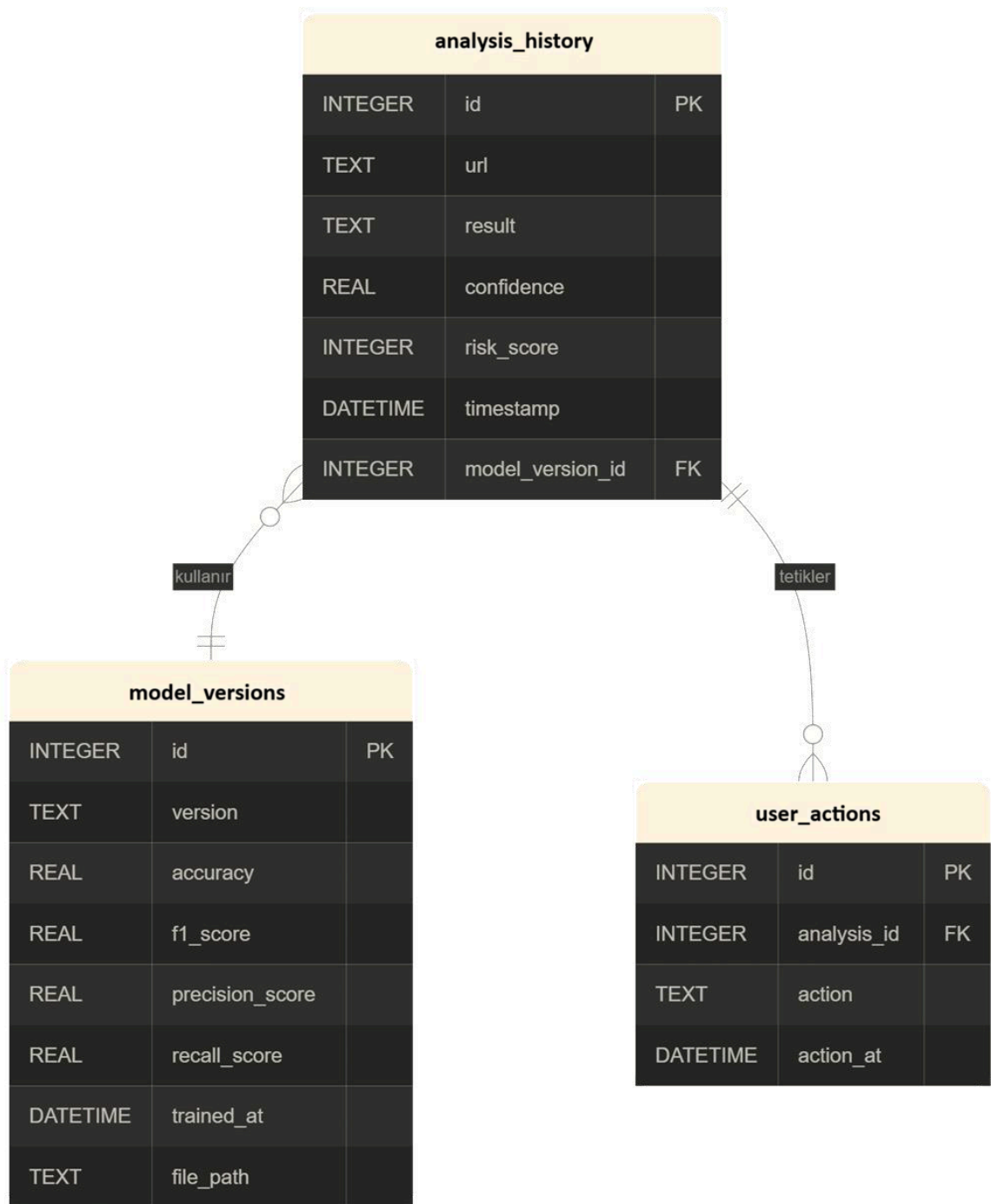
Sütun Adı	Veri Tipi	Kısıtlar	Açıklama
id	INTEGER	PK, AUTOINCREMENT	Her analiz kaydı için benzersiz tanımlayıcı. Otomatik artırılır.
url	TEXT	NOT NULL	Analiz edilen web sitesinin tam URL adresi.
result	TEXT	NOT NULL	Model çıktısı: 'safe', 'suspicious' veya 'phishing' değerlerinden biri.
confidence	REAL	NOT NULL, 0.0-1.0	Modelin tahmin güven skoru. 0.0 (en düşük) ile 1.0 (en yüksek) arasında.
risk_score	INTEGER	NOT NULL, 0-100	Güven skorunun 0-100 ölçeğine dönüştürülmüş hali.
timestamp	DATETIME	DEFAULT NOW	Analiz işleminin gerçekleştiği tarih ve saat. Otomatik atanır.

Tablo oluşturma SQL ifadesi aşağıdaki gibidir:

```
CREATE TABLE IF NOT EXISTS analysis_history (  
  id      INTEGER PRIMARY KEY AUTOINCREMENT,  
  url     TEXT    NOT NULL,  
  result  TEXT    NOT NULL CHECK(result IN ('safe','suspicious','phishing')),  
  confidence REAL  NOT NULL CHECK(confidence >= 0.0 AND confidence <= 1.0),  
  risk_score INTEGER NOT NULL CHECK(risk_score >= 0 AND risk_score <= 100),  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Performans optimizasyonu amacıyla timestamp sütununa indeks eklenecektir: CREATE INDEX idx_timestamp ON analysis_history(timestamp DESC). Bu indeks, son analizleri tarihe göre hızlı sorgulamayı sağlar.

C.5.2. E/R Diyagramı



C.5.3. Stored Procedures ve Triggers

PHOENIX projesinin veri tabanı tasarımı, sade ve işlevsel bir yapıyı hedeflemektedir. Sistemin mevcut gereksinimleri standart SQL INSERT ve SELECT ifadeleriyle karşılanabilmekte; karmaşık stored procedure veya trigger mekanizmalarına ihtiyaç duyulmamaktadır.

Bununla birlikte, veri bütünlüğünü korumak amacıyla aşağıdaki basit kural uygulanacaktır:

- **Kayıt Sınırlama:** Veri tabanındaki kayıt sayısı 1000'i aştığında en eski 100 kayıt otomatik olarak silinecektir. Bu işlem Flask API katmanında Python kodu ile yönetilecek; veri tabanı düzeyinde trigger yerine uygulama düzeyinde kontrol tercih edilmiştir.
- **Veri Doğrulama:** result sütunundaki CHECK kısıtı yalnızca 'safe', 'suspicious' ve 'phishing' değerlerine izin vermekte; geçersiz veri girişini veri tabanı seviyesinde engellemektedir.

C.6. Donanım Tasarımı

Bu proje kapsamında özel bir donanım tasarımı gerçekleştirilmemiştir. PHOENIX sistemi, mevcut bilgisayar donanımları üzerinde yazılım tabanlı olarak çalışacak şekilde tasarlanmıştır.

C.6.1. Sistem Mekanik Bileşen Tasarımlarının Verilmesi

Bu proje kapsamında mekanik bileşen içeren bir sistem tasarımı bulunmamaktadır. PHOENIX sistemi tamamen yazılım tabanlı olarak geliştirilmiş olup herhangi bir fiziksel veya mekanik bileşen içermemektedir.

C.6.2. Elektronik Tasarım

Bu proje kapsamında elektronik devre veya donanım tabanlı bir tasarım gerçekleştirilmemiştir. PHOENIX sistemi tamamen yazılım bileşenlerinden oluşmakta olup herhangi bir elektronik tasarım içermemektedir.

C.6.3. Gömülü Yazılım Tasarımları

Bu proje kapsamında gömülü sistemlere yönelik bir yazılım geliştirilmemiştir. PHOENIX sistemi, genel amaçlı bilgisayarlar üzerinde çalışan yazılım tabanlı bir uygulama olup gömülü yazılım tasarımı içermemektedir.